

ARCHON AGENTIC OS 2026 — VOLLSTÄNDIGE FEATURELISTE

System- & Modul-Katalog, Architektur-Übersicht und Funktionsmatrix

Stand: 11. November 2026 | Version: v0.5.1 („Archon Hardened“) | Sprache: Deutsch

System- & Modul-Katalog, Architektur-Übersicht und Funktionsmatrix

© Michael Keller

Inhalt

1. Versionsstand & Release-Übersicht (v0.5.1) 3
2. System-Ansichten & Spezial-Studios (UI Views) 3
3. Token-Spar- & FinOps-Ökosystem (Kostenminimierung) 8
4. Kernel, Scheduling & Daemon-Architektur (Pillar 1) 11
5. Multi-Agenten-Prozessmodell, Hierarchie & Synthese (Pillar 2) 12
6. MCP-Treiber, Externe Dienste & Multimodale Werkzeuge (Pillar 3) 14
7. Gedächtnis, Datenbank-Hygiene & Obsidian 2nd Brain (Pillar 4) 15
8. Sicherheit, Zero-Trust Governance & System-Diagnose (Pillar 6) 17
9. Selbst-Evolution, Marktplätze & Auto-Dokumentation (Pillar 7) 19
10. Referenz-Matrix: Steuerbare Evey/Hermes-Plugins 19
11. Enterprise- & Resilienz-Architektur 21
12. Autonomie-, Sandbox- & Governance-Architektur 23
13. Performance-Core, Durchsatz-Skalierung & Token-Effizienz 24
14. Moderne Gedächtnis- & RAG-Architektur 25
15. Zuverlässige Ausführung & Determinismus (Durable Execution) 27
16. Sicherheit, Datenschutz & Enterprise-Guardrails 28
17. Selbst-Optimierung, Evaluation & Schwarm-Intelligenz 29
18. Säule 3: Wirtschaft & ERP (alpha) & Mandanten-Suite 31
19. Architektur-, Security- & Deployment-Features (v0.5.0 – v0.5.1) 31
20. Alphabetisches Glossar & Stichwortverzeichnis 32

© Michael Keller

1. Versionsstand & Release-Übersicht (v0.5.1)

Version	Codename	Wesentliche Architektur- & Feature-Meilensteine
v0.5.1	Archon Hardened	Modulare FastAPI-Router (main.py -88%), 25 kritische + 68 hohe Security-Befunde behoben, Docker-Admin-Proxy Klasse A, RBAC-Benutzerverwaltung (PBKDF2), 2-Phasen Systemupdate (?local_only=true), AnythingLLM Proxy-Integration, portabler 1-Click-Installer.
v0.5.0	Wirtschaft & ERP (alpha)	Vollständige Säule 3 (CRM/PocketBase-Sync, Artikel/Leistungen, Vorgänge ANG/AB, Rechnungswesen mit DATEV & GiroCode, Corporate-Mandanten), Research Lab v2 & 55 Boardroom-Frameworks.
v0.3.x	Sentinel / Foundation	7-Säulen OS-Kernel, 70 Spezialisten-Kader, 34+ Evey-Plugins, FinOps 3-Tier-Routing, Ephemeral Code-Sandboxen, Replay Debugging, PII-Scrubber, Semantic Cache & FTS5 Hybrid Search.

2. System-Ansichten & Spezial-Studios (UI Views)

1. Ansicht: Mission Control (Monitoring-Cockpit & Leitstand)

Problem: Ohne zentrales Echtzeit-Cockpit fehlen der Überblick über aktive Agenten, Token-Durchsatz, Tageskosten und blockierte Freigaben.

Mechanik: Bündelt 5 vitale KPI-Kacheln (Flottenlast, Durchsatz TPM, Kostenwächter, Stabilität, Approval Gate), die 5-Spalten-Board-Projektion, die Active Process Inspection Bar (3-Slot-Footer für Heartbeat, Tool-Concurrency und Context Saturation), das 15-Minuten-Token-Aktivitätsband (SVG-Sparkline) und den Live-Audit-Stream.

Nutzen: *Liefert eine verzögerungsfreie 360°-Überwachung des gesamten Betriebssystems auf einem einzigen Bildschirm.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

2. Ansicht: Operations – Kanban Board & Task-Engine

Problem: Aufgaben in reinen Chat-Verläufen gehen unter und lassen sich weder nach Status noch nach Verantwortlichkeiten steuern.

Mechanik: 5 Lebenszyklus-Spalten (To-Do, In Bearbeitung, Review & QA, Blockiert, Abgeschlossen) direkt auf kanban.db (Single Source of Truth). Unterstützt konfigurierbare WIP-Limits pro Spalte, Prioritätsfilter (P0–P3), Agenten-Zuweisung, Zeiterfassung und 1-Klick-Obsidian-Verknüpfung. Integriert das 4-Augen Qualitäts-Gate: Vor Verschieben einer Aufgabe auf 'done' prüft ein Peer-Reviewer (Katharina Völk für QM/Compliance oder Robert) das Arbeitsergebnis auf Vollständigkeit, Substanz und Handlungsfähigkeit (1–100% Score). Bei 'changes_requested' greift eine Status-Sperre; ein Human-in-the-Loop Override (force=True) übersteuert bei Bedarf.

Nutzen: *Strukturiert alle operativen Projekte transparent und verhindert Systemüberlastung durch Work-in-Progress-Schranken.* Verhindert unfertige Arbeitsstände im 'done'-Status durch zwingendes Vier-Augen-Review.

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

3. Ansicht: Operations – Interaktives 2D-Organigramm & Berichtslinien

Problem: Flache Listen von Agenten zeigen keine Weisungsbefugnisse oder Eskalationspfade bei blockierten Prozessen.

Mechanik: Visualisiert 56 Knoten und 55 Berichtslinien als interaktiven 2D-Graphen (Apex -> CEO Robert -> Fach-Leads -> Spezialisten). Unterstützt Drag & Drop zum Umhängen von Vorgesetzten, Raster-Ausrichtung,

Team-Karten-Ansicht und dauerhafte Speicherung in config/org_hierarchy.json.

Nutzen: *Definiert verbindliche Eskalations- und Delegationsketten, denen das System bei Blockaden automatisch folgt.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

4. Ansicht: Operations – Interne Agenten-Kommunikation (Funkverkehr)

Problem: Wenn Spezialisten untereinander Teilaufgaben übergeben, überflutet dies entweder den Nutzer-Chat oder bleibt unsichtbar.

Mechanik: Dedizierte Abhörzentrale mit Kanalliste (#allgemein, #tech-dev, #legal, #marketing) und chronologischem Nachrichten-Feed aller Inter-Agenten-Absprachen aus kanban.db.

Nutzen: *Macht die Zusammenarbeit der Agenten-Flotte nachvollziehbar, ohne das primäre Nutzer-Postfach zu belasten.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

5. Ansicht: Eingang & Triage (3-Spalten HITL-Gatekeeper & Postfach)

Problem: Unkontrollierte Schreibaktionen von Agenten (z. B. E-Mail-Versand) sowie verstreute Eingänge aus Mail und Telegram bergen Sicherheitsrisiken.

Mechanik: Dreispaltige Triage-Zentrale: Spalte 1 für gelb markierte Human-in-the-Loop-Freigaben (Freigeben/Ablehnen), Spalte 2 für IMAP/SMTP-E-Mail-Synchronisation mit Auto-Triage & Delegation, Spalte 3 für den gespiegelten Live-Telegram-Stream (@Beatman_bot).

Nutzen: *Stellt sicher, dass keine externe Schreiboperation ohne ausdrückliche Freigabe des Betreibers ausgeführt wird.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

6. Ansicht: Agenten-Kader & Flottenmanagement (8-Reiter-Cockpit)
Problem: Spezialisten benötigen individuelle Modelle, System-Prompts, Werkzeuge und Arbeitsanweisungen an einem zentralen Ort.
Mechanik: Verwaltet 70 Spezialisten-Profile mit Schnellfiltern (Aktiv, Favoriten, Alle, 7 Domänen), Hot-Swap für Provider & Modell sowie 8 Fach-Reitern: Übersicht (360°-Dossier & Telegram-Rufname), Direkt-Chat, Fähigkeiten (Skills), Operations (Task-Historie), SOUL (System-Prompt-Editor), Personalakte (SOPs), Talent-Marktplatz und Skill-Marktplatz.
Nutzen: Zentrale Personalabteilung zur sekundenschnellen Kalibrierung und Erweiterung des gesamten Agenten-Stamms.
Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

7. Ansicht: Master Chat (Dual-Mode Workspace & Multi-Session-Sidebar)

Problem: Beim direkten Arbeiten mit Agenten fehlte eine fokussierte Vollbild-Umgebung mit schnellem Wechsel zwischen 1:1-Dialog, Gruppenchat und Missions-Auftrag.

Mechanik: Bietet den Dual-Mode-Switch ('Direkt' für sofortige Antwort vs. 'Mission' für strukturierte Task-Anlage), Multi-File-Upload (bis 5 Dateien), Browser-Screen-Capture (Kamera-Button), Spracheingabe (Diktat), Reasoning-Budget-Wähler (0 bis 16k Tokens), KI-Prompt-Optimizer und rechte Session-Historie (Agenten vs. Gruppe). Beinhaltet die Telegram Voice-Pipeline: Eingehende Sprachnachrichten (.ogg/Opus) von unterwegs werden via Whisper/Gemini transkribiert, anhand von Rufnamen (/robert, /anwalt etc.) an den Fachagenten

geroutet und Antworten via edge-tts (de-DE-ConradNeural) als natürliche Sprachmemos direkt auf Telegram zurückgesendet.

Nutzen: *Vereint Hochgeschwindigkeits-Dialog, multimodale Eingabe und Missions-Steuerung in einer durchgängigen Arbeitsfläche.* Ermöglicht vollständige mobile Sprachbedienung und bidirektionalen Audio-Austausch über Telegram.

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

8. Ansicht: AI Mastermind & Boardroom (55 Methodik-Frameworks)

Problem: Komplexe strategische Entscheidungen erfordern strukturierte Management-Methoden und kontroverse Perspektiven statt Einzelmeinungen.

Mechanik: Simuliert mehrstufige Gremiensitzungen (Chairman + 3 reale Fachagenten-Slots + Red-Team-Challenger) anhand von 34 auswählbaren Frameworks in 7 Kategorien (35 Base) (z. B. CLEAR, Design Sprint, Double Diamond, Lean Startup, First Principles, Game Theory, Blue Ocean, Pre-Mortem, Six Sigma).

Nutzen: Erzwingt methodische Strenge, deckt blinde Flecken durch das Red-Team auf und liefert entscheidungsreife Board-Memos.

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

9. Ansicht: Showroom (Projekt-Explorer, Artefakt-Galerie & Python-Sandbox)

Problem: Generierte Berichte, HTML-Prototypen, Kalkulationen und Skripte gehen in langen Konversationen verloren.

Mechanik: Katalogisiert alle Arbeitsergebnisse nach Ersteller, Datum und Typ. Beinhaltet den Workspace Vault Reader mit Live-Rendering für Markdown/HTML/JSON sowie eine isolierte Python-Sandbox zur sicheren Code-Ausführung. Erweitert um den interaktiven Showroom Live-Runner: Ermöglicht die direkte Ausführung von Python- und Code-Dateien im Browser mit 12s Timeout, isoliertem Scratch-Verzeichnis und ausklappbarem Terminal-Drawer (STDOUT, STDERR, Exit-Code und Millisekunden-Laufzeit).

Nutzen: *Dient als Abnahme- und Präsentationshalle für alle fertigen Deliverables vor dem Kunden- oder Produktionseinsatz.* Code-Artefakte und Skripte lassen sich ohne Konsolenwechsel direkt im Showroom verifizieren und testen.

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

10. Ansicht: Zentralbibliothek & 2nd Brain (RAG, Auto-OCR & Obsidian-Vault)

Problem: Fachbücher, DIN-Normen und Bebauungspläne als PDF konnten von Agenten bisher nicht ohne manuelles Kopieren durchsucht werden.

Mechanik: Vereint 3 Sub-Tabs und 4 Core Hubs (00_HR_Agenten, 02_Wissensbibliothek, Protokolle/, 00_Notebook_Synthesen): 1. AnythingLLM RAG-Index (Port 3001), 2. Dokumenten-Erfasser mit Drag & Drop, Auto-OCR (Docling) und KI-Themenklassifikation, 3. Direkter Obsidian-Dateibaum & Markdown-Viewer inklusive Vault-Umschalter.

Nutzen: Verwandelt PDF-Gutachten vollautomatisch in durchsuchbares Markdown- und Vektor-Wissen für alle 70 Agenten.

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

11. Ansicht: Workflow-Engine (n8n Automation Mesh auf Port 5678)

Problem: Wiederkehrende Schnittstellenprozesse (Cron-Jobs, Webhooks, Datenbank-Abgleiche) verbrauchen unnötig Tokens, wenn sie über LLMs laufen.

Mechanik: Betten den visuellen n8n-Workflow-Editor direkt ein und verbindet ihn über den Skill n8n-automation-mesh (FastMCP) mit dem CEO-Agenten Robert. Aktivierte 0-Token-Routinen: 1. Morning-Briefing (werktags 07:30 Uhr mit Obsidian-Ablage und Telegram-Zustellung), 2. Weekly Maintenance (sonntags 23:00 Uhr mit SQLite-Backup & Health-Audit), 3. RSS & News Watchdog (stündlicher Scan), plus 5 aktive Funktions-Webhooks für Web-Extraktion, Messaging, Dokumenten-Rendering, Feed-Suche und CRM-Sync.

Nutzen: *Kombiniert deterministische 0-Token-Hintergrundprozesse mit kognitiven Agenten-Entscheidungen an Schlüsselstellen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

12. Ansicht: LLM Gateway (OmniRoute AI Router Dashboard)

Problem: Die feste Bindung an einen einzigen Cloud-Anbieter führt bei Rate-Limits oder Ausfällen zum sofortigen Systemstillstand.

Mechanik: Zentrales Proxy-Dashboard (Port 20128 / 8000) mit Multi-Provider-Vault, Echtzeit-Latenzmessung (ms), HTTP-Fehlermonitor (429/500), automatischem Circuit-Breaker und 4-stufiger Fallback-Kaskade (GLM-4.7 Flash -> Gemini 2.5 Flash -> Claude 3.5 Sonnet -> lokales Llama 3.3 70B).

Nutzen: *Sichert 99,9 % Verfügbarkeit aller kognitiven Prozesse bei minimalen API-Kosten.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

13. Ansicht: Token Guard (Guardian & Token Analytics in 3 Sub-Views)

Problem: Fehlende Transparenz darüber, welcher Agent, welches Modell oder welche Einzelsitzung die meisten Tokens und Kosten verursacht.

Mechanik: Bietet Zeitfilter (24h, 7d, 30d, Gesamt), Inaktivitäts-Timer und 3 Sub-Views: 1. Übersicht & Tages-Trend-Chart mit Profil-/Modell-Tabellen und Schutz-Schranken-Formular, 2. Einzel-Session Verbrauchs-Inspektor (Input, Output, Cache-Read Tokens sortierbar), 3. Ressourcen & Task-Drosselung (Schieberegler 1–16 Tasks).

Nutzen: *Volle Kostenkontrolle bis auf die einzelne Session-ID herab inklusive sofortiger Budget- und Parallelitäts-Begrenzung.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

14. Ansicht: Erweiterungen & Modul-Zentrale (34 Evey-Spezial-Plugins)

Problem: Sicherheits-, Speicher- und Analysefunktionen ließen sich bisher nicht modular pro Fachgebiet zu- oder abschalten.

Mechanik: Steuert 34 spezialisierte Hermes/Evey-Plugins über 8 Fach-Reiter: Sicherheit & Wächter (4), Performance & Cache (2), Monitoring & Analytics (3), Agenten-Intelligenz (6), Planung & Tasks (2), Brücken & Schnittstellen (4), Gedächtnis & Wissen (3), Tools & Execution (3) plus Finanzen/Qualität/Wartung. Beinhaltet das standardisierte Pluginsystem & Runtime-Hooks: Modulare Ordnerstruktur (plugins/<plugin_id>/plugin.json), 4 Lifecycle-Hooks (on_load, on_enable, on_disable, on_unload) und 2 Tool-Interzeptoren (before_tool_execute, after_tool_execute) mit vorinstallierten Starter-Plugins (System Metrics, Security Audit Hook, Cost Guard).

Nutzen: *Erlaubt das präzise Zu- und Abschalten einzelner Systemfähigkeiten im laufenden Betrieb per Mausklick.* Ermöglicht isolierte Runtime-Erweiterungen und globale Tool-Interzeption ohne Eingriff in den Kernel-Code.

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

15. Ansicht: Research Lab (Deep Search & Evidenz-Triangulation)

Problem: Einfache Suchanfragen liefern ungeprüfte Einzeltreffer ohne Quellenabgleich oder Vertrauensbewertung.

Mechanik: Mehrstufiges Forschungs-Cockpit mit wählbarer Suchtiefe, Quellen-Monitor, Evidenz-Score, Live-Synthese-Protokoll zur Triangulation widersprüchlicher Aussagen und 1-Klick-Export in den Obsidian-Vault. Beinhaltet die Autonome Tiefenrecherche & Reflexionsschleife v2 (Re-Search Loop): Zerlegung komplexer Recherchen in 3–5 Teilfragen, autonome Lückenanalyse, bis zu 3 iterative Nachfolge-Suchläufe, Multi-Provider Fallback (SearXNG / Jina Reader) und SSRF-Schutz.

Nutzen: *Erzeugt belastbare, zitierfähige Markt-, Rechts- und Technik-Dossiers.* Schließt Wissenslücken selbstständig durch rekursive Folgerecherchen bei widersprüchlichen oder unvollständigen Quellen.

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

16. Ansicht: Telemetrie & Logs (3-Quellen Live-Audit Stream)

Problem: Bei Fehlern in Containern oder Agenten-Prompts musste bisher mühsam im Terminal über Docker-Befehle gesucht werden.

Mechanik: Bündelt 3 Log-Quellen in Echtzeit: Physische Docker-Container-Logs (stdout/stderr der 11 Container), kognitive Agenten-Logs und den kryptografischen Audit-Trail. Mit Level-Filter (DEBUG, INFO, WARN, ERROR), Auto-Scroll-Lock und Export.

Nutzen: *Unbestechlicher Flugschreiber für sekundenschnelle Fehlerdiagnose und Compliance-Nachweise.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

17. Spezial-Studio: Archon Prompt Studio (832 Vorlagen & Fabric SOP)

Problem: Unpräzise formulierte Aufgaben führen zu Halluzinationen, falschen Ausgabeformaten und teuren Korrekturschleifen.

Mechanik: Integrierter Baukasten mit 824 Enterprise-Vorlagen in 16 Fachkategorien, über 255 Fabric-SOP-Mustern und der 5-Felder-Formel (1. Rolle, 2. Kontext, 3. Aufgabe, 4. Format, 5. Constraints/Negativ-Regeln) sowie KI-Prompt-Optimizer.

Nutzen: Zwingt jede Delegation in eine glasklare Struktur und überträgt sie per Klick in Chat oder Kanban-Aufgabe.

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

18. Spezial-Studio: Visual CSS Live Studio & In-Place Transform Gizmo (Alt + S)

Problem: Design-Anpassungen erforderten bisher das Suchen in CSS-Dateien und ständiges Neuladen des Browsers.

Mechanik: Schwebendes Echtzeit-Design-Studio (Alt + S) mit Fadenkreuz-DOM-Inspektor, interaktivem Transform-Gizmo (Position ziehen, 4 Ecken/Kanten skalieren, Eckenrundung ziehen), Theme-Profil-Manager und direktem CSS-Export nach custom_studio.css.

Nutzen: *Ermöglicht pixelgenaues Live-Styling der gesamten Benutzeroberfläche im laufenden Betrieb ohne Seiten-Reload.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

19. Spezial-Studio: Design System Lab, AppShell Architekt & 15-Komponenten-Studio

Problem: Abweichende Header-Höhen, Abstände oder Kontraste zwischen Hell- und Dunkelmodus stören die visuelle Ruhe.

Mechanik: Beinhaltet den interaktiven AppShell-Architekten (56px Header-Flucht, Sidebar-Breite, Viewport-Padding mit Live-Miniatursimulation) sowie das 15-Komponenten-Design-Studio (os-workspace bis os-badge) mit Dual-Theme-Engine (Light/Dark).

Nutzen: *Garantiert ein einheitliches, blendfreies B2B-SaaS-Erscheinungsbild über alle 20 Ansichten hinweg.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

20. Spezial-Ansichten: System Control Center (11 Fachreiter, Live-Dock) & Globale Omnibox (Strg + K)
Problem: Einstellungen waren verstreut; die Navigation zwischen 20 Ansichten und 70 Agenten erforderte viele Mausklicks.
Mechanik: Das System Control Center bündelt 7 Administrations-Reiter (Hermes Core inkl. YAML-Editor, Container & Dienste, Provider & Gateway, Methodiken, Plugins, Kommunikation, Design) und lässt sich per 'Live-Dock' rechts andocken. Die Omnibox (Strg + K) navigiert per Tastatur sofort zu jedem Tab (#), Agenten (@) oder Befehl. Integriert das 1-Klick Updatesystem & Continuous Deployment: Automatischer GitHub-Sync mit Pre-Update Snapshots aller SQLite-Datenbanken (WAL) in data/snapshots/ vor jedem Git-Pull, 1-Klick Rollback und minimalen Git-Stat-Prüfungen gegen Mount-Timeouts.
Nutzen: Maximale Bediengeschwindigkeit für Administratoren und Power-User. Ermöglicht gefahrlose Software-Updates mit automatischem Datenbank-Rollback im Fehlerfall.
Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

3. Token-Spar- & FinOps-Ökosystem (Kostenminimierung)

21. 3-Tier Automatische Kosten-Routing-Caps (LOW / MEDIUM / HIGH)

Problem: Wenn einfache Routineaufgaben auf teuren Spitzenmodellen (wie Claude Sonnet oder GPT-4o) laufen, explodieren die Tageskosten.

Mechanik: Konfigurierbare Preis-Obergrenzen pro 1 Million Tokens im System Control Center: LOW (max. $0.10/1M -> erzwingt Free-Tier/Flash-Lite), MEDIUM (max. $1.00/1M -> Haiku/Gemini Flash), HIGH (No Cap -> beste Reasoning-Modelle für komplexe Architektur).

Nutzen: *Senkt die laufenden API-Ausgaben im Alltag um bis zu 85 %, ohne bei schwierigen Aufgaben an Qualität einzubüßen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

22. FreeLLM-API Docker-Proxy & 'auto/best-free' Routing (Port 3002)

Problem: Kostenlose Kontingente verschiedener Anbieter sind schwer manuell zu überwachen und wechseln häufig.

Mechanik: Ein dedizierter Docker-Container (freellmapi auf Port 3002) bündelt kostenfreie LLM-Endpunkte. Das Routing-Profil 'auto/best-free' wählt automatisch das stärkste verfügbare Null-Euro-Modell.

Nutzen: *Ermöglicht den Dauerbetrieb von Hintergrund- und Triage-Agenten zu exakt $0.00 API-Kosten.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

23. Stufenlose Reasoning-Budget-Steuerung im Master Chat & Router (0 bis 16k Tokens)

Problem: Reasoning-Modelle (Thinking-Mode) verbrauchen oft tausende unsichtbare Denk-Tokens für triviale Fragen.

Mechanik: 6-stufiger Regler (Aus: 0 Tok, Minimal: ~1k, Niedrig: ~2k, Mittel: ~4k, Hoch: ~8k, Maximal: ~16k) im Chat-

Footer und in agent_router.py.

Nutzen: *Schaltet teure Denk-Schleifen bei einfachen Aufgaben komplett ab und reserviert tiefes Reasoning für echte Komplexfälle.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

24. Evey-Cache & Cache-TTL-Steuerung (24h Semantischer & Exakter Antwort-Cache)

Problem: Wiederkehrende identische oder sehr ähnliche Systemabfragen führten jedes Mal zu neuen, kostenpflichtigen API-Calls.

Mechanik: Das Plugin evey-cache speichert Modellantworten mit konfigurierbarer Lebensdauer (Standard: 24 Stunden TTL) und weist im Session-Inspektor die eingesparten Cache-Read-Tokens separat aus.

Nutzen: *Beantwortet wiederholte Abfragen in 0 Millisekunden zu 0 Token-Kosten.*

Preis & Schattenseite (Trade-off): Ist die Cache-TTL zu hoch (> 72h), liefert das System bei dynamischen Themen veraltete Antworten.

Wann ausschalten? & Schwellenwerte: Ausschalten bei Echtzeit-Abfragen (Live-Kurse, Serverstatus). Std-TTL: 24h.

25. Max-Context & Max-History Hard-Clipper im Guardian (32k / 20 Turns)

Problem: Endlos wachsende Chat-Sitzungen senden bei jeder neuen Frage die gesamte Historie erneut mit und treiben die Input-Tokens exponentiell nach oben.

Mechanik: Im Guardian einstellbare Hardschranken für das maximale Kontextfenster (z. B. 32.000 Tokens) und die maximale Anzahl übermittelter Chat-Nachrichten (z. B. letzte 20 Turns).

Nutzen: *Kapselt die maximal möglichen Input-Kosten pro Turn verlässlich nach oben ab.*

Preis & Schattenseite (Trade-off): Schneidet Nachrichten oberhalb von 32k Tokens oder 20 Turns hart ab, wodurch sehr früher Kontext wegfällt.

Wann ausschalten? & Schwellenwerte: Für tiefe Dokumenten-Analysen auf 64k–128k im Hermes Core anheben.

26. Dynamischer Token-Clipper für Low-Context-Modelle (backend/token_clipper.py)

Problem: Kompakte oder lokale Modelle (8k/16k) brechen ab, wenn große Tool-Ausgaben das Kontextfenster sprengen.

Mechanik: Bewahrt zwingend den System-Prompt (Index 0) und die letzte Benutzeranweisung, kürzt überlange Einzelblöcke intelligent und entfernt ältere Zwischenschritte.

Nutzen: *Erlaubt den stabilen Einsatz extrem günstiger Kleinmodelle auch bei umfangreichen Aufgaben.*

Preis & Schattenseite (Trade-off): Kürzt überlange Tool-Ausgaben in der Mitte heraus ([...SNIP...]), wodurch einzelne Code-Zeilen im Rohtext fehlen können.

Wann ausschalten? & Schwellenwerte: Kurzzeitig deaktivieren, wenn komplette Roh-Logs ungekürzt benötigt werden.

27. Sliding Window Memory Compression (Konsolidierter Entscheidungskontext)

Problem: Beim einfachen Abschneiden alter Nachrichten gehen frühere Projektbeschlüsse verloren.

Mechanik: Verdichtet ältere Gesprächsrunden zu einer kompakten Zusammenfassung und injiziert diese als Block [KONSOLIDIERTER GESPRÄCHS- & ENTSCHEIDUNGSKONTEXT] auf Position 1.

Nutzen: *Spart bis zu 70 % historische Konversations-Tokens bei vollem Erhalt aller getroffenen Entscheidungen.*

Preis & Schattenseite (Trade-off): Der Komprimierungs-Schritt kostet einmalig ~800 Tokens und glättet wörtliche Formulierungen früher Runden.

Wann ausschalten? & Schwellenwerte: Bei Bedarf über protect_last_n (Std: 20) im Hermes Core feinjustieren.

28. LRU-K Page Swapping für Subagenten-RAM (backend/context_swapper.py)

Problem: Parallele Subagenten halten riesige Mengen an Rohdaten gleichzeitig im aktiven Prompt-Speicher.

Mechanik: Lagert nicht mehr akut benötigte Kontext-Seiten ab 80 % Auslastung nach dem Least-Recently-Used-Prinzip in SQLite aus und lädt sie nur bei Bedarf (Re-Hydration) zurück.

Nutzen: *Hält die Prompts schlank und verhindert unnötige Token-Gebühren für inaktives Hintergrundwissen.*

Preis & Schattenseite (Trade-off): Das Auslagern und Nachladen von Agenten-RAM auf Disk erzeugt ~10–25 ms I/O-Latenz beim Kontextwechsel.

Wann ausschalten? & Schwellenwerte: Ab 80 % RAM-Auslastung aktiv lassen, um OOM-Abstürze zu verhindern.

29. JEV Router — 0-Token Deterministische Intent-Vorfilterung (jev_router.py)

Problem: Jede einfache Systemanweisung über ein großes Sprachmodell zu klassifizieren, verschwendet Tokens und Zeit.

Mechanik: Klassifiziert eingehende Eingaben vor dem LLM-Aufruf per deterministischem JSON Execution Vector und Keyword-Heuristik.

Nutzen: *Leitet klare Systembefehle direkt an die Zielroutine weiter, ohne ein einziges LLM-Token zu verbrauchen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

30. ASD-STE100 Simplified Technical English Prompter (.agents/skills/asd-ste100-prompter)

Problem: Weitschweifige Prosatexte in System-Prompts und Agenten-Anweisungen blähen jeden einzelnen API-Call unnötig auf.

Mechanik: Transformiert Instruktionen nach dem Luftfahrt-Standard ASD-STE100 (eine Bedeutung pro Wort, aktive Verben, kurze Sätze, null Füllwörter).

Nutzen: *Reduziert die Token-Länge von System-Prompts um ca. 30–40 % und senkt gleichzeitig die Fehlerrate bei Tool-Calls.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

31. Anti-AI-Slop & Output-Komprimierung (.agents/skills/anti-ai-slop)

Problem: Sprachmodelle erzeugen standardmäßig lange Höflichkeitsfloskeln, Einleitungen und redundante Zusammenfassungen (teure Output-Tokens).

Mechanik: Unterbindet KI-Floskeln, leere Phrasen und künstliches Aufblähen direkt im System-Prompt und bereinigt Ausgaben.

Nutzen: *Spart bis zu 25 % der besonders teuren Output-Tokens und liefert sofort verwertbare Fachtexte.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

32. Pre-Flight Kosten-Schätzer & Budget-Prognose (backend/cost_estimator.py)

Problem: Vor dem Start großer Aufgaben war nicht absehbar, was der Lauf kosten wird.

Mechanik: Berechnet vorab über POST /api/router/estimate-cost die erwarteten Input-/Output-Tokens sowie den Dollar-Betrag anhand der aktuellen Modell-Preisliste und gleicht ihn mit dem Task-Budget ab.

Nutzen: *Warnt vor teuren Läufen, noch bevor der erste Cent ausgegeben wird.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

33. Dynamisches Task-Budget mit 80%-Spar-Downgrade & 100%-Stopp

Problem: Ein einzelner komplexer Task konnte unbemerkt das gesamte Tagesbudget verzehren.

Mechanik: Überwacht die kumulierten Kosten pro Aufgabe (z. B. Limit $0.25): Schaltet bei 80 % automatisch auf ein günstiges Spar-Modell herunter und pausiert den Task bei 100 % mit Telegram-Freigabeanfrage.

Nutzen: *Verhindert Kostenausreißer bei mehrstufigen autonomen Recherchen.*

Preis & Schattenseite (Trade-off): Bei Erreichen von 80 % des Task-Budgets wird auf ein günstigeres Spar-Modell

heruntergeschaltet; bei 100 % erfolgt der Hard-Stop.

Wann ausschalten? & Schwellenwerte: Task-Budget (Std: 0,50 $) für große Architektur-Läufe auf 2,00 $ anheben.

34. Loop-Killer & Max-Steps-Begrenzung (15 Schritte / Jaccard > 90%)

Problem: Wenn ein Agent bei einem fehlerhaften Tool-Aufruf in eine Endlosschleife gerät, verbrennt er im Sekundentakt Guthaben.

Mechanik: Prüft jeden Schritt auf exakte Wiederholungen (>= 3) oder semantische Kreisbezüge (> 90 % Ähnlichkeit) und kappt Aufgaben spätestens nach dem konfigurierbaren Max-Steps-Limit (Standard: 15 Schritte).

Nutzen: *Schützt das Konto zuverlässig vor unbemerkten Endlosschleifen über Nacht.*

Preis & Schattenseite (Trade-off): Zu scharfe Grenzwerte (Max Identisch = 2 oder Jaccard < 0.80) stoppen legitime iterative Aufgaben wie schrittweises Test-Fixing oder Paginierung zu früh.

Wann ausschalten? & Schwellenwerte: Kurzzeitig ausschalten, wenn ein Agent bewusst viele gleichförmige Schleifen-Schritte (z. B. Massen-Migration von 50 Dateien) abarbeiten soll. | Schwellenwerte: Max Identisch: 2 = stoppt Retries | > 5 = teure Schleifen (Std: 3). Jaccard: < 0.80 = Fehlalarme | > 0.96 = erkennt Schleifen spät (Std: 0.90). Max Schritte: Std 15.

4. Kernel, Scheduling & Daemon-Architektur (Pillar 1)

35. Hermes Master Orchestrator (Hermes-Core)
Problem: Unkoordinierte Einzelagenten arbeiten gegeneinander oder verlieren das Gesamtziel aus den Augen.
Mechanik: Zentrale Prozess-Leitstelle zur Auftragsanalyse, Rollenauswahl, Überwachung und Ergebnis-Abnahme.
Nutzen: Sichert strategische Kohärenz über alle 70 Spezialisten.
Preis & Schattenseite (Trade-off): Mehr aktive Hintergrund-Dienste und hohes Reasoning ('high') erhöhen die Grundlast und Token-Kosten.
Wann ausschalten? & Schwellenwerte: Über die 14 Sektionen im Tab 'HERMES CORE' granular steuerbar.

36. Tickless Kernel Daemon & Asyncio Event-Loop (Port 8420 / 8501)

Problem: Ständiges Polling im Sekundentakt verschwendet CPU-Ressourcen und Systemenergie.

Mechanik: Berechnet den exakten Zeitpunkt des nächsten fälligen Tasks ('Sleep-until-next-due') und versetzt den Kernel bis dahin in den Ruhezustand.

Nutzen: *Minimale Systemlast im Leerlauf bei verzögerungsfreier Aufwach-Reaktion.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

37. Prioritäts-Preemption Scheduler (P0–P3 Warteschlange)

Problem: Dringende Benutzerfragen im Direkt-Chat mussten warten, wenn Hintergrund-Jobs alle Slots belegten.

Mechanik: Priorisierte Warteschlange: P3-Aufgaben (Direkt-Chat / Notfall) überholen laufende P1/P2-Routinejobs sofort.

Nutzen: *Garantiert schnelle Antwortzeiten im Dialog trotz hoher Hintergrundlast.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

38. Frei skalierbare Concurrency-Drosselung (backend/concurrency_throttle.py)

Problem: Zu viele gleichzeitige Agenten führen zu HTTP-429-Rate-Limits und 99 % CPU-Spitzen.

Mechanik: Reguliert aktive Worker über eine Asyncio-Semaphore (einstellbar per Slider von 1 bis 16 parallelen Tasks im Guardian und Control Center).

Nutzen: *Erlaubt gedrosselten Schutzbetrieb auf Laptops ebenso wie Hochlastbetrieb auf Workstations.*

Preis & Schattenseite (Trade-off): Knappe Stagnations-Limits (< 30 Sek.) brechen langsame Deep-Reasoning-Modelle (z. B. o3/R1) fälschlich mitten im Denkprozess ab; mehr Worker erhöhen RAM- und API-Spitzenlast.

Wann ausschalten? & Schwellenwerte: Ausschalten bei extrem langlaufenden Einzel-Analysen oder lokalen LLM-Läufen auf langsamer Hardware. | Schwellenwerte: Max Worker: 1–2 = Rückstau | > 8 = RAM-Spitzen & HTTP-429 (Std: 4). Stagnation: < 30s = bricht Denker ab | > 180s = späte Heilung (Std: 45s).

39. Autonomer 30-Sekunden-Watchdog & Crash-Recovery (watchdog_daemon.py)

Problem: Abgestürzte oder eingefrorene Tasks blieben dauerhaft im Status in_progress hängen.

Mechanik: Prüft alle 30 Sekunden sämtliche laufenden Prozesse, bricht Zombie-Tasks nach Timeout (120s) ab und überführt verwaiste Aufgaben in die automatische Crash-Recovery.

Nutzen: *Hält alle Worker-Slots frei von Karteileichen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

40. Deadlock- & Stagnations-Supervisor (backend/deadlock_supervisor.py)

Problem: Gegenseitige Abhängigkeiten zwischen zwei Agenten können zu einem stillen Einfrieren führen.

Mechanik: Erkennt zirkuläre Wartezustände und Tasks ohne Fortschritt (> 45s) und löst die Blockade autonom mit exponentiellem Backoff auf.

Nutzen: *Verhindert Pipeline-Stillstände bei komplexen Gruppenarbeiten.*

Preis & Schattenseite (Trade-off): Ein zu knappes Stagnations-Limit (< 30s) bricht langsame Deep-Reasoning-Modelle (o3/R1) mitten im Denkprozess ab.

Wann ausschalten? & Schwellenwerte: Bei Nutzung von Reasoning-Modellen Stagnations-Limit auf 90–120s erhöhen (Std: 45s).

41. Flotten-Selbstheilung & Container-Auto-Restart (backend/fleet_self_healing.py)

Problem: Fällt einer der 11 Docker-Container aus, musste er manuell neu gestartet werden.

Mechanik: Überwacht alle Core-Container über den abgesicherten docker-socket-proxy und führt bei Ausfällen (> 60s) autonom einen Neustart inklusive Audit-Eintrag durch.

Nutzen: *Sichert den unbeaufsichtigten 24/7-Betrieb der gesamten Docker-Landschaft.*

Preis & Schattenseite (Trade-off): Ein dauerhaft defekter Container kann durch Auto-Restart in eine Neustart-Schleife geraten.

Wann ausschalten? & Schwellenwerte: Während manueller Container-Wartungsarbeiten kurzzeitig deaktivieren.

42. Proaktiver Morgen-Standup & Weckdienst (backend/autonomous_standup.py)

Problem: Morgens fehlte eine kompakte Übersicht über nächtliche Läufe, Kosten und anstehende Entscheidungen.

Mechanik: Generiert täglich um 08:00 Uhr einen konsolidierten Statusbericht im Obsidian-Vault (01_PROJEKTE/Daily_Briefings/) und pusht ihn an Dashboard und Telegram.

Nutzen: *Startet jeden Arbeitstag mit einer fertigen Entscheidungsvorlage.*

Preis & Schattenseite (Trade-off): Jedes automatische Morgen-Briefing verbraucht ~1.500–3.000 Tokens zum eingestellten Zeitpunkt.

Wann ausschalten? & Schwellenwerte: Ausschalten, wenn kein täglicher Morgen-Report gewünscht ist (Std: 08:00 Uhr).

5. Multi-Agenten-Prozessmodell, Hierarchie & Synthese (Pillar 2)

43. 70 Deklarative Agenten-Profile & Markdown-Personalakten
Problem: Generische KI-Assistenten haben kein Rollenbewusstsein und keine definierten Qualitätsstandards.
Mechanik: Jeder der 70 Spezialisten ist in agent_profiles.json sowie einer eigenen Personalakte (00_HR_Agenten/) mit

System-Prompt, Modellroute, Werkzeug-Scope und SOPs verankert.
Nutzen: Schafft verlässliche virtuelle Fachkräfte für Strategie, Recht, Bau, Finanzen, Marketing und IT.
Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

44. Robert Delegation, Fan-in-Synchronisation & Lisa Marker-Kontrakt (SYNTHESE-BEREIT)

Problem: Bei parallelen Teilaufgaben gingen Teilergebnisse verloren oder wurden unvollständig zusammengefasst.

Mechanik: Robert zerlegt Großprojekte; ein Prozess-Lock (_MC_DATA_LOCK) wartet auf den Abschluss aller Teilaufgaben (Fan-in). Lisa aggregiert die Berichte unter Prüfung des Pflicht-Markers SYNTHESE-BEREIT; Robert erteilt in Zeile 1 die formale FREIGABE.

Nutzen: *Mehrstufige Qualitätssicherung nach dem Vorbild einer echten Unternehmensführung.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

45. Standardisierter Agent-to-Agent Handoff-Desk (backend/handoff_desk.py)

Problem: Stille Kontextverluste bei der Übergabe von einem Spezialisten an den nächsten.

Mechanik: Erzeugt formale Übergabeprotokolle mit Deliverables, offenen Annahmen und SHA-256-Prüfsumme, die vom Empfänger-Agenten quittiert werden müssen.

Nutzen: *Revisionssichere Schnittstellen zwischen allen Abteilungen.*

Preis & Schattenseite (Trade-off): Strikte SLA-Prüfungen weisen sehr kurze, aber korrekte Faktenantworten zurück, wenn min_content_length zu hoch (> 300) ist.

Wann ausschalten? & Schwellenwerte: Standard: 100 Zeichen Mindestlänge, 60 Min. Turnaround.

46. Zweistufiger DAG-Scheduler für parallele Projekt-Ausführung (backend/dag_scheduler.py)

Problem: Sequentielle Abarbeitung vieler Teilaufgaben dauert bei Großprojekten zu lange.

Mechanik: Berechnet den gerichteten azyklischen Graphen (DAG): Führt unabhängige Recherchen (Stufe 1) parallel aus und startet danach automatisch die Synthese (Stufe 2).

Nutzen: *Verkürzt die Durchlaufzeit komplexer Analysen um ein Vielfaches.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

47. Self-Correction & Tool Execution Reflection Loop

Problem: Flüchtige Syntax- oder Parameterfehler bei Tool-Aufrufen führten zum sofortigen Abbruch.

Mechanik: Fängt Tool-Fehler ab, reflektiert die Fehlermeldung und korrigiert die Aufrufparameter autonom in bis zu 3 Versuchen.

Nutzen: *Deutlich höhere Robustheit bei Datei-, Datenbank- und Web-Operationen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

48. Unterbrechbare Langzeit-Workflows mit SQLite State-Snapshotting

Problem: Wartet ein Task stundenlang auf eine Nutzer-Freigabe, belegt er unnötig Speicher oder verliert seinen Zustand bei einem Neustart.

Mechanik: Friert den exakten Ausführungszustand in der Tabelle workflow_checkpoints ein und nimmt die Arbeit nach Freigabe nahtlos wieder auf.

Nutzen: *Ermöglicht mehrtägige Geschäftsprozesse über Neustarts hinweg.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert

(vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

49. Dual-Agent Konsens-Gate & Adversarial Critic Review (backend/consensus_gate.py)

Problem: Ein einzelner Agent bemerkt eigene Denkfehler oder Halluzinationen bei kritischen Aufgaben oft nicht.

Mechanik: Erzwingt bei P3-Tasks und kritischen Eingriffen ein formales Gegenlesen durch einen unabhängigen Revisor-Agenten (Red-Team/Critic). Im Kanban-Board als automatisiertes 4-Augen Qualitäts-Gate verdrahtet (Katharina Völk / Robert Review mit 1–100% Score und Status-Lock bei Revisionsbedarf).

Nutzen: *Vier-Augen-Prinzip für alle geschäftskritischen Ergebnisse.*

Preis & Schattenseite (Trade-off): Verdreifacht (3x) den Token-Verbrauch und erhöht die Antwortzeit um +6 bis +20 Sek.; Einstimmigkeit birgt Deadlock-Gefahr.

Wann ausschalten? & Schwellenwerte: Erst ab Priorität P3/P4 aktivieren und 2/3-Mehrheit wählen.

6. MCP-Treiber, Externe Dienste & Multimodale Werkzeuge (Pillar 3)

50. Archon CRM & PocketBase-Ökosystem (Ports 3005, 8090, 8092)

Problem: Kundenkontakte, E-Mail-Historien und Medien-Assets lagen getrennt von den KI-Agenten.

Mechanik: Dediziertes CRM-Frontend (Port 3005) mit PocketBase-Backend (Port 8090) und Media-Server (Port 8092), voll angebunden über scripts/archon_crm_mcp.py.

Nutzen: *Agenten können Kontakte abrufen, nachrecherchieren, bewerten (Lead-Scoring) und Korrespondenz zuordnen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

51. Automatisches E-Mail-Matching & LinkedIn-Archiv-Importer

Problem: Tausende Kontakte aus LinkedIn-Exporten und E-Mail-Postfächern mussten manuell abgeglichen werden.

Mechanik: Synchronisiert Postfächer (--fetch-mails), ordnet Mails automatisch bestehenden CRM-Kontakten zu und importiert komplette LinkedIn-Connections-CSVs inklusive Deduplizierung.

Nutzen: *Baut in Minuten eine vollständige, angereicherte Beziehungsdatenbank auf.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

52. Robert Tool-Permission Manager (backend/tool_permission_manager.py)

Problem: Agenten sollen bei fehlenden Python-/Node-Paketen nicht hilflos abbrechen, dürfen aber auch nichts ungefragt installieren.

Mechanik: Robert stellt einen formellen Installationsantrag (pending_permission) mit Befehl und Begründung; erst nach deiner 1-Klick-Freigabe wird das Paket installiert.

Nutzen: *Kontrollierte Erweiterbarkeit bei vollem Schutz des Host-Systems.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

53. Event-Driven Webhook-Mesh (backend/event_mesh.py)

Problem: Externe Ereignisse aus GitHub, Formularen oder Drittsystemen konnten keine Agenten-Aktionen auslösen.

Mechanik: Universeller Webhook-Empfänger, der externe JSON-Payloads validiert und direkt in ausführbare Kanban-Tasks umwandelt.

Nutzen: *Verbindet Archon OS in Echtzeit mit der restlichen IT-Landschaft.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

54. Zentraler Multi-Channel Notification Hub (backend/notification_hub.py)

Problem: Wichtige Warnungen gingen im UI unter, während unwichtige Meldungen auf dem Handy störten.

Mechanik: Routet Meldungen strikt nach Schweregrad: INFO -> UI-Toast, WARNING -> Dashboard-Log, CRITICAL -> sofortiger Telegram-Push.

Nutzen: *Zielgenaue Alarmierung ohne Benachrichtigungs-Müdigkeit.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

55. Multi-Modale Vision- & Screen-Capture-Pipeline (backend/vision_pipeline.py)

Problem: Visuelle Probleme oder Tabellen in Bildern mussten mühsam beschrieben werden.

Mechanik: Direkte Bildschirmaufnahme im Browser (Kamera-Button im Chat) sowie automatische OCR- und Strukturanalyse für hochgeladene Bilder (.png, .jpg, .webp).

Nutzen: *Agenten sehen genau das, was du auf dem Bildschirm siehst.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

56. Apollo Voice Engine — Edge-TTS & Whisper Diktat (apollo_voice_engine.py)

Problem: Lange Texte am Smartphone oder Desktop einzutippen kostet Zeit; Berichte möchte man sich unterwegs anhören.

Mechanik: Bidirektionale Sprachpipeline: Mikrofon-Diktat (Speech-to-Text) direkt im Chat-Eingabefeld und natürliche Sprachausgabe (Text-to-Speech) für Briefings. Vollständige Telegram-Voice-Integration (POST /api/telegram/voice-relay und POST /api/voice/synthesize) für mobile Audio-Memos via edge-tts (de-DE-ConradNeural).

Nutzen: *Verwandelt Archon OS in einen sprachgesteuerten Executive Assistant.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

57. Oracle News Radar & SEO Execution Agent

Problem: Marktveränderungen und SEO-Schwachstellen eigener Webseiten wurden nur sporadisch geprüft.

Mechanik: Oracle News Radar scannt kontinuierlich Fachquellen auf Projektrelevanz; der SEO Agent prüft Seiten auf Meta-Tags, Struktur und Keyword-Dichte.

Nutzen: *Kontinuierliche Markt- und Sichtbarkeitsoptimierung im Hintergrund.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

7. Gedächtnis, Datenbank-Hygiene & Obsidian 2nd Brain (Pillar 4)

58. Zentraler SQLite WAL-Memory-Layer (5 Standardtabellen)

Problem: Gleichzeitige Lese- und Schreibzugriffe mehrerer Agenten führten bei einfachen Dateien zu Konflikten.

Mechanik: Transaktionale SQLite-Datenbank (data/agentic_memory.db) im Write-Ahead-Logging-Modus (WAL) mit den 5 Kerntabellen tasks, conversations, learnings, kv_store und user_profile.

Nutzen: *Blockadefreie Parallelzugriffe mit höchster Datensicherheit.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

59. Prozesssicheres Kanban-Sidecar mit atomarem Dateiaustausch (backend/kanban_sidecar.py)

Problem: Metadaten wie WIP-Limits oder Tags konnten bei parallelen Schreibvorgängen korrumpieren.

Mechanik: Schützt config/kanban_meta.json über einen prozessweiten Mutex und atomare Write-Replace-Operationen.

Nutzen: *Verhindert Datenverlust bei hochfrequenten Board-Aktualisierungen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

60. Docker-RAG-Bridge zu AnythingLLM (5.068 indexierte Dokumente auf Port 3001)

Problem: Das im Docker-Vektorspeicher vorhandene Wissen wurde bei normalen Chat-Anfragen oft nicht automatisch herangezogen.

Mechanik: backend/rag_bridge.py verbindet den Agentenkern direkt mit dem AnythingLLM-Workspace ARCHON und liefert relevante Textpassagen als Grounded Context.

Nutzen: *Fundierte Antworten auf Basis deiner echten Unternehmensdokumente statt allgemeiner Schätzungen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

61. Dreistufige Projektordner-Disposition im Obsidian-Vault

Problem: Wenn Agenten Dateien erzeugen, entsteht ohne feste Regeln schnell Unordnung im Vault.

Mechanik: Legt für jedes Projekt automatisch die 3-Stufen-Struktur an: 01_Zwischenberichte, 02_Medien und 03_Endprodukte.

Nutzen: *Saubere Trennung zwischen Rohentwürfen und abgenommenen Endergebnissen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

62. Obsidian Vault Indexer & Automatische Map of Content (backend/vault_hygiene.py)

Problem: Neue Projektberichte blieben im Obsidian-Graph oft unverlinkt ('Orphan Notes').

Mechanik: Scannt den Vault, spürt verwaiste Notizen auf und aktualisiert vollautomatisch die zentrale _MOC_Projekt_Index.md mit Wikilinks.

Nutzen: *Hält den Wissensgraphen in Obsidian jederzeit vollständig vernetzt.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

63. Autonomes Fact-Harvesting & 85%-Ähnlichkeits-Deduplizierung (backend/post_task_pipeline.py)

Problem: Wichtige Erkenntnisse aus abgeschlossenen Aufgaben gingen unter oder wurden mehrfach doppelt gespeichert.

Mechanik: Extrahiert nach Task-Abschluss automatisch neue Fakten für learnings und user_profile und verhindert Dubletten ab 85 % Textähnlichkeit.

Nutzen: *Das System wird mit jeder erledigten Aufgabe klüger, ohne den Speicher zuzumüllen.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

64. Automatisches Online-DB-Backup mit 7-Tage-Rotation (backend/db_backup_manager.py)

Problem: Ohne regelmäßige Datenbanksicherungen droht bei Hardwaredefekten der Verlust aller Tasks und Profile.

Mechanik: Führt im laufenden Betrieb PRAGMA wal_checkpoint(TRUNCATE) aus und speichert komprimierte .db.gz-Archive in .hermes/backups/db/.

Nutzen: *Lückenlose Datensicherung ohne eine Sekunde Ausfallzeit.*

Preis & Schattenseite (Trade-off): Tägliche Voll-Backups belegen je nach Aufbewahrungsdauer ~50–500 MB Festplattenspeicher.

Wann ausschalten? & Schwellenwerte: Standard: 7 Tage Aufbewahrung (db_backup_keep_days).

65. Autonomer SQLite Vacuum & Fragmentierungs-Optimizer (backend/db_optimizer.py)

Problem: Gelöschte Datensätze und häufige Updates blähen SQLite-Dateien über Monate unnötig auf.

Mechanik: Führt über POST /api/system/optimize-db auf allen Datenbanken PRAGMA optimize und VACUUM aus und berechnet den freigegebenen Speicherplatz.

Nutzen: *Hält Abfragezeiten dauerhaft im Millisekundenbereich.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

66. Atomares Pre-Write-Snapshotting & 1-Klick-Rollback (backend/snapshot_manager.py)

Problem: Wenn ein Agent eine Datei fehlerhaft überschreibt, war die Wiederherstellung des Vorzustands aufwendig.

Mechanik: Sichert vor jedem Schreibzugriff automatisch das Original in .hermes/snapshots/ und stellt es per Klick oder API sofort wieder her. Zusätzlich als Pre-Update Snapshots aller SQLite-Datenbanken vor GitHub-Pulls und Container-Updates in data/snapshots/ integriert.

Nutzen: *Vollständiges Sicherheitsnetz bei allen Datei- und Code-Änderungen.*

Preis & Schattenseite (Trade-off): Jeder Pre-Write-Snapshot kostet ~5–15 ms Disk-I/O vor dem Überschreiben einer Datei.

Wann ausschalten? & Schwellenwerte: Ringpuffer standardmäßig auf 50 Snapshots begrenzt.

67. Git-kompatible Live-Diff-Vorschau (backend/diff_engine.py)

Problem: Vor einer Freigabe möchte man exakt sehen, welche Zeilen sich ändern, statt die ganze Datei neu zu lesen.

Mechanik: Erzeugt einen farbigen Zeilenvergleich (Rot = entfernt, Grün = neu) zwischen Bestand und Agenten-Entwurf.

Nutzen: *Sekundenschnelle Prüfung von Code- und Dokumentenänderungen im HITL-Gate.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

8. Sicherheit, Zero-Trust Governance & System-Diagnose (Pillar 6)

68. Kryptografisch verkettetes SHA-256 Audit-Log (backend/audit_log.py)

Problem: Einfache Text-Logs lassen sich nachträglich verändern oder löschen.

Mechanik: Jedes Ereignis wird mit dem SHA-256-Hash des Vorgängereintrags verkettet (Blockchain-Prinzip) und ist über GET /api/audit/verify auf Manipulation prüfbar.

Nutzen: *Revisionssicherer Nachweis aller System- und Freigabeaktionen.*

Preis & Schattenseite (Trade-off): Minimaler Schreib-Overhead (~2 ms pro Eintrag); die kryptografische Kette wächst über Monate an.

Wann ausschalten? & Schwellenwerte: Für revisionssichere Governance dauerhaft eingeschaltet lassen.

69. Local Security Guard — 25 Prompt-Injection- & Jailbreak-Signaturen

Problem: Bösartige E-Mails oder Webseiten können versteckte Befehle enthalten ('Ignore previous instructions...').

Mechanik: Scannt jeden externen Texteingang anhand von 25 Angriffsmustern, bevor er in den Kontext eines Agenten gelangt.

Nutzen: *Wehrt indirekte Prompt-Injections an der Systemgrenze ab.*

Preis & Schattenseite (Trade-off): Sehr strenge Heuristiken können bei IT-Security-Texten oder Pen-Testing-Prompts Fehlalarme (False Positives) auslösen.

Wann ausschalten? & Schwellenwerte: Nur für autorisierte Red-Teaming-Tests kurzzeitig lockern.

70. Secret-Masking & Red-Teaming Output Guard (backend/secret_masker.py)

Problem: API-Schlüssel oder Passwörter könnten versehentlich in Logs, Berichten oder Chat-Ausgaben auftauchen.

Mechanik: Filtert ausgehende Texte in Echtzeit auf Schlüssel-Muster (sk-..., Bearer, Passwörter) und ersetzt sie durch [GESCHÜTZTER_KEY].

Nutzen: *Verhindert das versehentliche Offenlegen vertraulicher Zugangsdaten.*

Preis & Schattenseite (Trade-off): Aggressive Regex-Maskierung kann in seltenen Fällen harmlose Base64- oder Hash-Strings in Code-Ausgaben schwärzen.

Wann ausschalten? & Schwellenwerte: Im Produktivbetrieb zum Schutz von API-Keys dauerhaft aktiv lassen.

71. Abgesicherter Docker-Socket-Proxy (Port 2375 intern)

Problem: Direkter Zugriff auf /var/run/docker.sock gibt Containern uneingeschränkte Root-Rechte über das Host-System.

Mechanik: Schaltet einen restriktiven Proxy-Container dazwischen, der nur explizit erlaubte Status- und Neustart-Befehle durchlässt.

Nutzen: *Schützt das Host-Betriebssystem nach dem Zero-Trust-Prinzip.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

72. A/B-Modell-Benchmarker & Empirischer Performance-Scorer (backend/model_benchmarker.py)

Problem: Neue Sprachmodelle erscheinen wöchentlich; ihre Eignung für eigene Unternehmensaufgaben bleibt oft unklar.

Mechanik: Testet zwei Modelle parallel mit identischen Aufgaben und vergleicht Antwortzeit, Token-Kosten und Ergebnisqualität.

Nutzen: *Objektive Datenbasis für die Modellwahl im Agenten-Kader.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

73. Systemweiter 360°-Health-Score & Diagnose-Engine (backend/system_diagnostics.py)

Problem: Einzelne Metriken zeigen oft nicht auf einen Blick, wie gesund das Gesamtsystem wirklich ist.

Mechanik: Berechnet aus Docker-Flottenstatus (35%), Concurrency-Queue (20%), Audit-Integrität (20%), Deadlock-Freiheit (15%) und RAG-Erreichbarkeit (10%) einen Gesamtscore von 0–100 % inkl. Handlungsempfehlungen.

Nutzen: *Sofortige Gesamtdiagnose des Betriebssystems über GET /api/system/health-score.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

9. Selbst-Evolution, Marktplätze & Auto-Dokumentation (Pillar 7)

74. Autonome Skill-Synthese mit Personalakten-Rückkopplung (backend/skill_synthesizer.py)

Problem: Erfolgreiche mehrstufige Lösungswege mussten bisher manuell als neuer Skill programmiert werden.

Mechanik: Erkennt wiederkehrende erfolgreiche Tool-Ketten, generiert automatisch einen SKILL.md-Entwurf, trägt ihn in die Personalakte des Agenten ein und legt ihn zur Freigabe vor.

Nutzen: *Das System erweitert seine Fähigkeiten im Betrieb selbstständig.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

75. Dynamischer Talent- & Skill-Marktplatz (skill_markt.js & AgentSkillsHub)

Problem: Neue Rollen oder Workflows für neue Geschäftsfelder von Grund auf neu zu schreiben, ist zeitintensiv.

Mechanik: Durchsuchbarer Katalog zum 1-Klick-Import und -Export kompletter Agenten-Profile sowie externer Skills (inkl. agentskillshub.dev Integration).

Nutzen: *Skaliert die Agenten-Flotte bei neuen Projekten in Sekunden.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

76. Vollständige Frontend-Backend-Synchronisation (238 Fetches / 581 Pfade / 29 Modale)

Problem: In großen Web-Anwendungen führen verwaiste Buttons oder unverbundene Modale oft zu stillen Fehlern.

Mechanik: Automatisierte Prüf-Pipeline stellt sicher, dass alle 238 Frontend-API-Aufrufe und 29 Modale exakt mit den 368 Backend-Routen verdrahtet sind.

Nutzen: *Garantiert 0 tote Links und 100 % bedienbare Steuerelemente in allen Ansichten.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

77. Autonomer Master-Dossier- & Handbuch-Generator (backend/system_dossier_generator.py)

Problem: Architektur-Dokumentationen veralten meist schon wenige Tage nach ihrer Erstellung.

Mechanik: Generiert auf Knopfdruck (POST /api/system/generate-dossier) das aktuelle System-Architektur-Dossier simultan im Projektordner und im Obsidian-Vault (00_DOKUMENTATION/).

Nutzen: *Hält die Systemdokumentation ohne manuellen Pflegeaufwand dauerhaft auf dem neuesten Stand.*

Preis & Schattenseite (Trade-off): Als permanente Kern-Ansicht bzw. System-Infrastruktur im Dashboard integriert (vernachlässigbarer RAM-/CPU-Grundbedarf; Token-Kosten entstehen nur bei aktiver Ausführung von LLM-Aktionen innerhalb des Moduls).

10. Referenz-Matrix: Steuerbare Evey/Hermes-Plugins

Nr. & Plugin-ID	Fach-Kategorie	Kernmechanik & Funktion im Betriebssystem
78. evey-cost-guard	Sicherheit & Wächter	Echtzeit-Kostenüberwachung und harte Budget-Abschaltung pro Tag und Task.
79. evey-session-guard	Sicherheit & Wächter	Überwacht Sitzungslaufzeiten, Token-Obergrenzen und verhindert Zombie-Sessions.
80. evey-email-guard	Sicherheit & Wächter	Scannt eingehende E-Mails vor der Triage auf Phishing und Prompt-Injections.

81. evey-watchdog	Sicherheit & Wächter	Prüft im 30s-Takt aktive Prozesse und startet blockierte Worker automatisch neu.
82. evey-cache	Performance & Cache	24h-Antwort-Cache zur sofortigen, kostenfreien Beantwortung wiederholter Abfragen.
83. disk-cleanup	Performance & Wartung	Bereinigt temporäre Artefakte, veraltete Logs und gibt Festplattenspeicher frei.
84. evey-telemetry	Monitoring & Analytics	Erfasst Token-Ströme, Latenzen und Modell-Aufrufe für Mission Control und Guardian.
85. evey-status	Monitoring & Analytics	Kontinuierlicher Health-Ping auf alle 11 Docker-Container und Gateways.
86. evey-delegation-score	Monitoring & Analytics	Bewertet die Erfolgs- und Qualitätsquote von Aufgabenübergaben zwischen Agenten.
87. evey-autonomy	Agenten-Intelligenz	Steuert den Autonomiegrad bei mehrstufigen Hintergrund-Entscheidungen.
88. evey-council	Agenten-Intelligenz	Mehrheits- und Konsensbildung zwischen mehreren Spezialisten (Mastermind-Kopplung).
89. evey-reflect	Agenten-Intelligenz	Selbstreflexions-Schleife zur eigenständigen Fehlerkorrektur vor Ergebnisabgabe.
90. evey-learner	Agenten-Intelligenz	Extrahiert dauerhafte Erkenntnisse aus erledigten Aufgaben für die learnings-Tabelle.
91. evey-identity	Agenten-Intelligenz	Sichert die strikte Einhaltung von SOUL.md, Tonalität und Rollen-Leitplanken.
92. evey-habits	Agenten-Intelligenz	Verwaltet wiederkehrende Routinen wie das tägliche 08:00-Uhr-Standup-Briefing.
93. evey-proactive	Agenten-Intelligenz	Erlaubt Agenten, bei erkannten Engpässen selbstständig Handlungsvorschläge zu machen.
94. evey-goals	Planung & Tasks	Verknüpft operative Einzelaufgaben im Kanban mit übergeordneten Unternehmenszielen.
95. evey-scheduler	Planung & Tasks	Zeit- und prioritätsgesteuerte Einplanung von Hintergrund-Aufträgen.
96. evey-bridge	Brücken & Schnittstellen	Interne Datendrehscheibe zwischen FastAPI-Monolith, Hermes-Daemon und Sidecars.
97. evey-mqtt	Brücken & Schnittstellen	Leichtgewichtiges Nachrichtenprotokoll für IoT- und Echtzeit-Statussignale.
98. evey-moltbook	Brücken & Schnittstellen	Synchronisiert strukturierte Arbeitsbücher und Synthese-Notizen.
99. evey-telegram-ux	Brücken & Schnittstellen	Steuert Rufnamen-Routing (/robert, /anwalt) und interaktive Freigabe-Buttons in Telegram.
100. evey-github	Brücken &	Anbindung für Repository-Operationen, Commit-Analysen

	Schnittstellen	und Issue-Tracking.
101. evey-research	Tools & Recherche	Tiefensuche-Treiber für Web-Scraping und Quellen-Extraktion im Research Lab.
102. evey-news	Tools & Recherche	Hintergrund-Scanner für Branchen-Feeds und Markt-Neuigkeiten (Oracle News Radar).
103. evey-digest	Tools & Recherche	Komprimiert umfangreiche Dokumentensammlungen in kompakte Management-Summaries.
104. evey-sandbox	Tools & Execution	Isolierte Ausführungsumgebung für generierten Python- und Analyse-Code im Showroom.
105. evey-rag	Gedächtnis & Wissen	Vektor-Retrieval-Treiber zur Anbindung von AnythingLLM und lokalen Wissensarchiven.
106. evey-memory-adaptive	Gedächtnis & Wissen	Dynamische Gewichtung von Kurzzeit- und Langzeitgedächtnis je nach Kontextdruck.
107. evey-memory-consolidate	Gedächtnis & Wissen	Nachtlauf zur Zusammenführung und Deduplizierung verwandter Gedächtniseinträge.
108. evey-wallet	Finanzen & Budgets	Verwaltet API-Guthaben, Kontingente und kostenbezogene Transaktions-Limits.
109. evey-validate	Qualität & Ausführung	Prüft Ausgaben auf Schema-Konformität, Pflicht-Marker (SYNTHESE-BEREIT) und Formatvorgaben.
110. evey-verification	Qualität & Ausführung	Faktencheck und Gegenprüfung kritischer Aussagen vor der Endfreigabe.
111. evey-delegate-model	Qualität & Ausführung	Wählt bei Teilaufgaben automatisch das kosten- und leistungsoptimale Zielmodell.

11. Enterprise- & Resilienz-Architektur

78. Replay & Time-Travel Debugging (backend/time_travel_debugger.py)

Problem: Bei komplexen Agenten-Workflows mit vielen Tool-Calls ist die nachträgliche Fehlerdiagnose im Textlog unübersichtlich und das Verzweigen von Ausführungssträngen unmöglich.

Mechanik: Speichert jeden Teilschritt mit Ein-/Ausgaben, Snapshots und Tokenverbrauch in 'execution_traces'. Ermöglicht interaktives Replay (GET /api/traces/{task_id}), State-Forking ab Schritt N (POST /api/traces/{task_id}/fork) und 1-Klick-Rollback modifizierter Dateien.

Nutzen: Schnelle Ursachenanalyse und risikofreie Was-wäre-wenn-Experimente mit alternativen Prompts oder Modellen ohne Neustart der gesamten Pipeline.

Preis & Schattenseite (Trade-off): Erhöht Disk-I/O um ~5–15 ms pro Datei-Edit und belegt je nach Snapshot-Limit ~50–250 MB zusätzlichen Speicherplatz auf der Festplatte.

Wann ausschalten? & Schwellenwerte: Ausschalten auf Systemen mit extrem knappem Disk-Speicher oder bei massenhaften Datei-Generierungen (z. B. Build-Artefakte). | Schwellenwerte: Max Snapshots: < 10 = kurze Historie | > 200 = höherer Disk-Verbrauch (Std: 50). Max Trace-Steps: Std 100.

79. Multi-Modell Eval-Matrix & A/B-Benchmarker (backend/model_benchmarker.py)

Problem: Modell-Zuweisungen an Agenten basieren mangels empirischer Vergleichswerte oft auf subjektiven Annahmen.

Mechanik: Sendet denselben Prompt simultan an zwei konfigurierte LLMs (POST /api/router/benchmark), misst Latenz, Token-Verbrauch und Antwortlänge und speichert die Historie persistent in 'model_benchmarks' (GET /api/router/benchmark/history).

Nutzen: Objektive, datengestützte Modellauswahl je Agentenrolle (z. B. DeepSeek V3 für Schnelligkeit/Kosten vs. Claude 3.7 für komplexe Architekturen).

Preis & Schattenseite (Trade-off): N Kandidaten-Modelle kosten N-fache API-Tokens pro Testlauf.

Wann ausschalten? & Schwellenwerte: Wann ausschalten? Nur gezielt für Modellvergleiche starten, nicht im Dauerbetrieb.

80. Tri-Agent Konsens-Gate & Schiedsgericht (backend/consensus_gate.py)

Problem: Kritische Eingriffe (Code-Refactorings, Migrationen) bergen bei Einzelentscheidungen das Risiko von Halluzinationen oder Sicherheitslücken.

Mechanik: Beruft ein dreiköpfiges Gremium (System Architect, Security Officer, QA Lead) ein (POST /api/consensus/council-vote). Jeder Revisor votiert unabhängig; Freigabe erfordert 2/3-Mehrheit. Alle Beschlüsse werden in 'consensus_decisions' archiviert (GET /api/consensus/history).

Nutzen: Verlässliche Risikominimierung und Eliminierung von Single-Point-of-Failure-Halluzinationen bei sensiblen Systemänderungen.

Preis & Schattenseite (Trade-off): Verdreifacht (3x) den Token-Verbrauch und erhöht die Antwortzeit um +6 bis +20 Sek. Bei Einstimmigkeits-Zwang droht Entscheidungs-Stillstand (Deadlock).

Wann ausschalten? & Schwellenwerte: Ausschalten (oder erst ab Priorität 4 = Kritisch aktivieren), wenn Geschwindigkeit und niedrige API-Kosten Vorrang haben. | Schwellenwerte: Ab Priorität: Prio 2 = fast jeder Task kostet 3x Tokens | Prio 4 = nur kritische Tasks (Std: Prio 3). Mehrheit: 2/3-Mehrheit empfohlen.

81. Autonome Task-Selbstheilung mit Git-Rollback (backend/task_self_healer.py)

Problem: Syntaxfehler oder fehlerhafte Code-Imports blockieren Tasks sofort und erfordern manuelles Eingreifen des Nutzers.

Mechanik: Isoliert den Stacktrace, führt bei Laufzeitfehlern einen Git-Rollback modifizierter Dateien durch und übergibt dem Agenten einen strukturierten Korrektur-Prompt (POST /api/healing/attempt). Bis zu 3 autonome Reparaturversuche; Log-Abruf via GET /api/healing/logs.

Nutzen: Maximale Resilienz bei Code- und Skript-Ausführungen mit automatischer Selbstreparatur vor der Eskalation ins HITL-Board.

Preis & Schattenseite (Trade-off): Jeder Selbstheilungsversuch kostet 1 zusätzlichen LLM-Call (+3 bis +12 Sek. Latenz & Token-Kosten). Ein automatisches Git-Rollback verwirft bei Fehlschlag auch nützliche Teilerfolge.

Wann ausschalten? & Schwellenwerte: Git-Rollback ausschalten, wenn unfertige Code-Entwürfe zur manuellen Weiterarbeit im Arbeitsverzeichnis stehen bleiben sollen. | Schwellenwerte: Max Reparaturversuche: 1 = gibt sofort auf | > 5 = teure Reparatur-Kaskade (Std: 3).

82. Event-Mesh & Posteingangs-Verzeichnis-Watcher (backend/event_mesh.py)

Problem: Aufgaben mussten bisher manuell über das Dashboard angestoßen oder über zeitgesteuertes IMAP-Polling abgefragt werden.

Mechanik: Zentraler Inbound-Event-Bus für externe Webhooks (POST /api/events/inbound/{channel}) und automatischer Verzeichnis-Watcher (POST /api/events/scan-directory), der neue Dateien in 'data/inbox/' direkt in Kanban-Triage-Tasks überführt.

Nutzen: Vollautomatische Event- und Belegerfassung aus externen Systemen (GitHub, n8n, Stripe, lokale Dateiscanner) ohne Benutzerinteraktion.

Preis & Schattenseite (Trade-off): Sehr kurze Scan-Intervalle (< 3 Sek.) erzeugen kontinuierliche Dateisystem-Polls; das tägliche Morgen-Standup verbraucht ~1.500–3.000 Tokens pro Lauf.

Wann ausschalten? & Schwellenwerte: Ausschalten, wenn der lokale Inbox-Ordner nicht aktiv genutzt wird oder kein morgendlicher Briefing-Bericht gewünscht ist. | Schwellenwerte: Scan-Intervall: 1–2s = unnötige Disk-Polls | > 30s = verzögerte Datei-Erkennung (Std: 5s). Standup-Zeit: Std 08:00 Uhr.

12. Autonomie-, Sandbox- & Governance-Architektur

83. Flüchtige Code-Sandboxen (backend/ephemeral_sandbox.py)

Problem: Ausführung von Agenten-Code birgt das Risiko von Systemabstürzen, Endlosschleifen oder unbeabsichtigten Dateiänderungen auf dem Host.

Mechanik: Führt Code in kurzlebigen, isolierten Docker-Micro-Containern mit harter Ressourcen-Begrenzung (512 MB RAM, 1 CPU, 30s Timeout, kein Netz) aus (POST /api/sandbox/execute). Die Sandbox wird nach Abschluss rückstandslos gelöscht.

Nutzen: Absolute Sicherheit und Systemstabilität bei der automatisierten Code-Ausführung und -Erprobung.

Preis & Schattenseite (Trade-off): Container-Kaltstart kostet ~300–800 ms Zusatzlatenz pro Tool-Call. Ist das Netzwerk deaktiviert, schlagen pip install oder externe API-Abrufe im Skript sofort fehl.

Wann ausschalten? & Schwellenwerte: Ausschalten (oder Netzwerk aktivieren), wenn vertrauenswürdige lokale Skripte direkten Host-Zugriff oder schnelle Paketinstallationen benötigen. | Schwellenwerte: RAM-Limit: < 256 MB = OOM-Kill bei Pandas/NumPy | > 2048 MB = Host-RAM-Risiko (Std: 512 MB). Timeout: < 10s = bricht Builds ab (Std: 30s).

84. Nächtliche Gedächtnis-Konsolidierung (backend/memory_consolidator.py)

Problem: Stetig anwachsende Traces, Heartbeats und veraltete Dialogdaten verlangsamen Datenbankabfragen im Dauerbetrieb.

Mechanik: Prunt in Ruhephasen (Standard 03:00 Uhr) veraltete Traces und unkritische Logs anhand konfigurierbarer Retention-Fristen (settings.json). Vollständig steuerbar über die Hermes Core Einstellungen (Uhrzeit, Retention-Tage, Pruning-Tiefe, manueller Run-Button).

Nutzen: Dauerhaft schlanke Datenbanken, reduzierte Latenzen und automatische System-Hygiene ohne manuellen Wartungsaufwand.

Preis & Schattenseite (Trade-off): Während des Konsolidierungslaufs (~10–30 Sek.) entsteht kurze CPU-/Disk-Last; eine zu niedrige Pruning-Schwelle (< 0.88) kann verwandte, aber eigenständige Fakten zusammenfassen.

Wann ausschalten? & Schwellenwerte: Ausschalten, wenn lückenlose Roh-Traces für ein forensisches Audit dauerhaft ungefiltert erhalten bleiben müssen. | Schwellenwerte: Aufbewahrung: < 7 Tage = Verlust wichtiger Audit-Spuren | > 60 Tage = große SQLite-DB (Std: 14 Tage). Pruning-Schwelle: 0.95 = konservativ | 0.88 = aggressiv (Std: 0.92).

85. Visueller Web-Agent mit Screenshot-Audit (backend/visual_web_agent.py)

Problem: Aufgaben auf komplexen Webportalen ohne API (SPAs, Formulare, interaktive Dashboards) können mit Text-Scrapern nicht bedient werden.

Mechanik: Steuert einen echten Headless-Chromium-Browser (POST /api/browser/browse-and-capture), erfasst Screenshots für jeden Klick- und Eingabeschritt in .hermes/visual_traces/{task_id}/ und extrahiert strukturierte Daten.

Nutzen: Lückenlose visuelle Nachvollziehbarkeit und automatisierte Interaktion mit modernen Webanwendungen ohne API-Zugang.

Preis & Schattenseite (Trade-off): Headless-Chromium benötigt pro aktivem Audit-Lauf ~300 MB RAM und ~3–5 Sek. Renderzeit.

Wann ausschalten? & Schwellenwerte: Wann ausschalten? Bei reinem Backend-Betrieb ohne UI-Prüfungen.

86. Virtueller War-Room & Pre-Mortem Stresstest (backend/war_room_simulator.py)

Problem: Unvorhergesehene Sicherheits-, Akzeptanz- oder Compliance-Risiken werden bei Produkteinführungen oft erst nach dem Rollout bemerkt.

Mechanik: Simuliert drei gegnerische Parteien (Kritischer Großkunde, Offensiver Pentester, Strenger Compliance-Auditor) vor der Freigabe (POST /api/simulation/war-room). Erstellt eine aggregierte Risiko-Matrix (LOW bis CRITICAL) mit verbindlichen Auflagen.

Nutzen: Proaktives Aufdecken von Schwachstellen und Vermeidung kostspieliger Fehlentscheidungen vor dem Livegang.

Preis & Schattenseite (Trade-off): Kostet ~1.500–3.000 zusätzliche Tokens und +4 bis +10 Sek. Wartezeit vor dem eigentlichen Task. Bei Block-Schwelle MEDIUM entstehen häufiger Fehlalarme.

Wann ausschalten? & Schwellenwerte: Auto-Pre-Mortem im normalen Entwicklungsalltag ausschalten und nur für

produktionskritische Eingriffe (HIGH/CRITICAL) nutzen. | Schwellenwerte: Block-Schwelle: MEDIUM = blockiert oft vorsorglich | CRITICAL = stoppt nur katastrophale Risiken (Std: HIGH).

87. SLA- & Übergabe-Verträge zwischen Agenten (backend/sla_contract_enforcer.py)

Problem: Lose Delegationen in Multi-Agenten-Hierarchien führen oft zu unvollständigen Zuarbeiten, die an den menschlichen Operator eskaliert werden.

Mechanik: Schließt bei internen Delegationen digitale Verträge mit Pflichtfeldern, Mindestlänge und Akzeptanzkriterien ab (POST /api/contracts/create). Validiert Lieferungen autonom (POST /api/contracts/validate) und weist mangelhafte Ergebnisse automatisch zur Nachbesserung ab.

Nutzen: Strikte Einhaltung von Qualitätsstandards im Agenten-Team und Vermeidung unfertiger Zwischenstufen im HITL-Board.

Preis & Schattenseite (Trade-off): Wird die Mindestlänge zu hoch angesetzt (> 300 Zeichen), werden präzise Kurzantworten abgelehnt und lösen teure Nacharbeits-Schleifen aus.

Wann ausschalten? & Schwellenwerte: Auto-Reject ausschalten, wenn Subagenten auch knappe Status- oder Ja/Nein-Antworten zurückgeben dürfen. | Schwellenwerte: Min. Länge: < 30 = lässt Platzhalter durch | > 300 = lehnt knappe Faktenantworten ab (Std: 100 Zeichen). Max Turnaround: Std 60 Min.

88. Zentrale Autonomie- & Schwellenwert-Steuerung (backend/autonomy_settings.py)

Problem: Feste Schwellenwerte im Programmcode verhindern eine schnelle Anpassung von Sicherheits-, Sandbox- und Resilienz-Regeln im laufenden Betrieb.

Mechanik: Bündelt alle 10 Autonomie-Gruppen (Concurrency, Loop-Killer, Selbstheilung, Docker-Sandbox, Time-Travel-Snapshots, Konsens-Gate, War-Room, SLA-Verträge, Inbox/Standup und Flotten-Hygiene) in settings.json und synchronisiert sie über GET/POST /api/settings/autonomy-modules mit dem System Control Center.

Nutzen: Jedes Schutz- und Autonomie-Modul lässt sich zur Laufzeit per Klick ein- oder ausschalten und exakt auf das gewünschte Risiko- und Ressourcenprofil feinjustieren.

Preis & Schattenseite (Trade-off): Ein defekter Container kann durch Auto-Restart in eine Crash-Loop geraten; der visuelle Browser-Audit startet Chromium (+300 MB RAM, +4 Sek. Dauer).

Wann ausschalten? & Schwellenwerte: Auto-Restart während manueller Container-Wartung ausschalten; Browser-Audit bei reinem Backend-Betrieb deaktivieren. | Schwellenwerte: DB-Backup-Tage: < 3 Tage = knappe Historie | > 30 Tage = ~500 MB+ Backup-Speicher (Std: 7 Tage).

13. Performance-Core, Durchsatz-Skalierung & Token-Effizienz

89. Semantisches Prompt- & Response-Caching (backend/semantic_cache.py)

Problem: Wiederkehrende oder leicht umformulierte Agenten-Anfragen verursachen unnötige LLM-API-Kosten und sekundenlange Wartezeiten.

Mechanik: Berechnet für jeden Prompt einen SHA-256-Hash sowie einen L2-normalisierten Wort-/3-Gramm-Vektor in SQLite und liefert bei einer Cosine-Ähnlichkeit >= 0.95 sofort den gespeicherten Response-Treffer.

Nutzen: Antwortzeiten unter 10 Millisekunden und 100 % Token-Ersparnis bei semantisch identischen Anfragen.

Preis & Schattenseite (Trade-off): Ist die Cosine-Schwelle zu niedrig (< 0.92), übersieht der Cache feine Bedeutungsunterschiede (z. B. 'lösche Test-DB' vs. 'lösche Prod-DB') und liefert falsche oder veraltete Antworten.

Wann ausschalten? & Schwellenwerte: Ausschalten bei Aufgaben mit Echtzeit-Daten (Börsenkurse, Live-Logs, Systemstatus) oder wenn jede Antwort frisch generiert werden muss. | Schwellenwerte: Cosine-Schwelle: < 0.92 = falsche Antworten aus Cache! | > 0.98 = kaum Treffer (Std: 0.95). TTL: Std 24h. Max Einträge: Std 500.

90. Paralleles Map-Reduce Sub-Tasking / Fan-Out & Fan-In (backend/map_reduce_executor.py)

Problem: Sequenzielle Abarbeitung vieler Einzeldokumente oder Teilfragen führt zu linearem Zeitwachstum und langen Wartezeiten.

Mechanik: Teilt Listen-Aufträge automatisch auf bis zu N parallele Asyncio-Worker (Semaphore-gesteuert) auf und führt die Teilergebnisse im Reduce-Schritt zusammen.

Nutzen: Vervielfacht den Durchsatz bei Dokumenten-, Recherche- und Code-Batch-Analysen ohne Überlastung der Event-Loop.

Preis & Schattenseite (Trade-off): Erzeugt N-fache gleichzeitige API-Last (Gefahr von HTTP-429 Rate-Limits beim Provider) und ~20–35 % Token-Overhead durch die anschließende Reduce-Synthese.

Wann ausschalten? & Schwellenwerte: Ausschalten bei strikten API-Rate-Limits oder wenn die Teilaufgaben eng voneinander abhängen und sequenziell aufeinander aufbauen müssen. | Schwellenwerte: Max Worker: > 8 = Rate-Limit-Gefahr (Std: 4). Chunk-Timeout: Std 30s. Ab Elementen: Std 3.

91. Speculative Fast-Routing & Latenz-Hedging (backend/speculative_hedger.py)

Problem: Einzelne langsame Cloud-Requests (Tail-Latency-Spikes) blockieren die gesamte Agenten-Ausführung.

Mechanik: Startet nach Ablauf von hedge_delay_seconds (z. B. 2.5s) automatisch einen parallelen Ersatz-Call auf einem schnellen Fallback-Modell und übernimmt das zuerst eintreffende Ergebnis.

Nutzen: Eliminiert Hänger durch langsame API-Endpunkte und stabilisiert die Antwortzeiten im 99. Perzentil.

Preis & Schattenseite (Trade-off): Wenn das Primärmodell kurz nach dem Hedge-Start doch noch antwortet, werden beide Requests abgerechnet (bis zu 2x Token-Kosten bei langsamen Prompts).

Wann ausschalten? & Schwellenwerte: Ausschalten bei knappem API-Budget oder wenn langsame Reasoning-Modelle genutzt werden, die regulär > 10 Sek. Denkzeit benötigen. | Schwellenwerte: Hedge-Verzögerung: < 1.5s = fast immer doppelte API-Kosten! | > 8s = kaum Latenz-Schutz (Std: 2.5s).

92. Dynamisches Context-Compaction & Hierarchisches Summarizing (backend/context_compactor.py)

Problem: Lange Agenten-Sitzungen blähen das Kontextfenster auf, verlangsamen den Prompt-Prefill und erhöhen die Kosten pro Turn drastisch.

Mechanik: Extrahiert ab einem einstellbaren Token-Schwellenwert alle harten Fakten (Ziele, Dateipfade, Entscheidungen, Fehler) in einen kompakten [HIERARCHICAL_MEMORY_STATE]-Block und behält nur die jüngsten Runden im Volltext.

Nutzen: Spart 65–80 % Kontext-Tokens bei langen Sessions ohne Verlust wichtiger Projektdaten.

Preis & Schattenseite (Trade-off): Der Komprimierungslauf kostet einmalig ~1.000 Tokens; exakte Code-Zeilen oder kleine Detailwerte aus weit zurückliegenden Nachrichten können in der Zusammenfassung verloren gehen.

Wann ausschalten? & Schwellenwerte: Ausschalten, wenn der Agent wortwörtlich auf sehr frühe Nachrichten derselben Sitzung zurückgreifen muss. | Schwellenwerte: Token-Schwelle: < 3000 = vergisst Details zu früh | > 24000 = hohe Prompt-Kosten (Std: 6000). Letzte Turns behalten: Std 4.

93. SQLite WAL Connection-Pool & Async Write-Batching (backend/async_write_batcher.py)

Problem: Hunderte einzelne SQLite-INSERTs bei parallelen Agenten-Läufen erzeugen Disk-I/O-Spitzen und 'database is locked'-Konflikte.

Mechanik: Sammelt Telemetrie- und Audit-Ereignisse in einem Thread-sicheren RAM-Ringpuffer und schreibt sie gebündelt via executemany() im WAL-Modus.

Nutzen: Reduziert Festplatten-Schreibzugriffe um rund 80 % und hält die Haupt-Event-Loop frei von Blockaden.

Preis & Schattenseite (Trade-off): Im Falle eines harten Stromausfalls oder kill -9 vor dem nächsten Flush-Intervall gehen die im RAM gepufferten Einträge der letzten 1–5 Sekunden verloren.

Wann ausschalten? & Schwellenwerte: Ausschalten, wenn jeder einzelne Log-Eintrag sofort und ohne Sekundenverzögerung synchron auf der Festplatte liegen muss. | Schwellenwerte: Batch-Größe: < 5 = viele Disk-Writes | > 100 = mehr Verlust bei Crash (Std: 25). Flush-Intervall: Std 2.0s.

14. Moderne Gedächtnis- & RAG-Architektur

94. Hybrid-Search mit BM25 FTS5 & Reciprocal Rank Fusion (backend/hybrid_search_engine.py)

Problem: Reine Vektorsuche findet semantisch ähnliche Texte, scheitert jedoch oft an exakten Fehlercodes, Funktionsnamen oder Seriennummern.

Mechanik: Kombiniert SQLite FTS5 Volltext-Ranking (BM25) und Cosine-Vektor-Ähnlichkeit über Reciprocal Rank

Fusion (RRF).

Nutzen: Vereint exakte Keyword-Treffer und semantisches Verständnis in einer einzigen, hochpräzisen Trefferliste.

Preis & Schattenseite (Trade-off): Doppelte Index-Abfrage erhöht die Such-Latenz um ~15–40 ms; ein zu hohes Top-K (> 10) füllt den Agenten-Prompt mit tausenden Zusatz-Tokens.

Wann ausschalten? & Schwellenwerte: Fast nie ausschalten — höchstens Top-K auf 3–5 reduzieren, wenn Token-Kosten pro Anfrage minimiert werden sollen. | Schwellenwerte: RRF-Konstante K: Std 60. BM25-Gewicht: 0.0 = nur Vektor | 0.5 = Balance | 1.0 = nur Stichwort (Std: 0.50). Top-K: Std 5.

95. Lokaler Knowledge-Graph / Tripel-Speicher (backend/knowledge_graph_store.py)

Problem: Flache Text-Chunks erfassen keine strukturierten Abhängigkeiten (wer verwaltet welches System oder welche Komponente nutzt welchen Dienst).

Mechanik: Extrahiert und speichert Subjekt-Prädikat-Objekt-Tripel mit Konfidenzscore in SQLite und ermöglicht mehrstufige Graph-Traversierungen (Multi-Hop).

Nutzen: Agenten erkennen System- und Organisationszusammenhänge über mehrere Ebenen hinweg.

Preis & Schattenseite (Trade-off): Automatische Tripel-Extraktion kostet beim Speichern ~500 Zusatz-Tokens. Ab > 2 Traversal-Hops explodiert die Anzahl der Nachbarknoten exponentiell.

Wann ausschalten? & Schwellenwerte: Auto-Extraktion ausschalten bei reinem Kurzzeit-Chat ohne langfristigen Wissensaufbau; Hops auf maximal 2 begrenzen. | Schwellenwerte: Max Hops: 1 = direkte Nachbarn | 2 = optimal | >= 3 = Knoten-Explosion! (Std: 2). Min. Konfidenz: Std 0.70.

96. Agentic Corrective RAG / CRAG (backend/corrective_rag.py)

Problem: Klassisches RAG reicht irrelevante Suchtreffer ungeprüft an das Sprachmodell weiter und provoziert dadurch Fehlantworten.

Mechanik: Bewertet jeden abgerufenen Dokument-Chunk gegen einen Relevanz-Schwellenwert (0.45) und führt bei schwachen Treffern automatisch ein Query-Rewrite mit Synonym-Expansion durch.

Nutzen: Filtert Wissensrauschen vor der Antwortgenerierung heraus und erhöht die Trefferqualität bei unklaren Fragen.

Preis & Schattenseite (Trade-off): Jede Query-Umformulierung kostet 1 zusätzlichen LLM-Call (+1,5 bis +4 Sek. Latenz). Ist die Relevanz-Schwelle zu hoch (> 0.70), dreht CRAG unnötige Extrarunden.

Wann ausschalten? & Schwellenwerte: Ausschalten, wenn die Wissensdatenbank klein und sauber gepflegt ist oder schnelle Antwortzeiten unter 2 Sekunden gefordert sind. | Schwellenwerte: Relevanz-Schwelle: < 0.30 = lässt schwache Quellen durch | > 0.70 = endlose Such-Schleifen (Std: 0.45). Max Rewrites: Std 2.

97. 2-Stufen Core-Memory nach MemGPT/Letta-Vorbild (backend/core_memory_manager.py)

Problem: Wichtige Nutzerpräferenzen und Projektziele gehen verloren, sobald sie aus dem kurzlebigen Chat-Fenster herausfallen.

Mechanik: Verankert editierbare Pinned-Working-Memory-Blöcke (<CORE_WORKING_MEMORY>) dauerhaft im System-Prompt, getrennt vom großen Archiv-Speicher.

Nutzen: Garantiert ein permanentes, vom Agenten selbst pflegbares Kerngedächtnis über beliebig viele Sitzungen hinweg.

Preis & Schattenseite (Trade-off): Jeder angepinnte Block belegt bei absolut jedem LLM-Aufruf dauerhaft Prompt-Tokens (10 Blöcke × 800 Zeichen ≈ 2.000 Tokens Grundlast pro Nachricht).

Wann ausschalten? & Schwellenwerte: Ausschalten oder Block-Anzahl auf 3–5 reduzieren, wenn kleine Modelle mit engem Kontextfenster oder knappe Token-Budgets genutzt werden. | Schwellenwerte: Max Blöcke: > 15 = hoher Token-Grundverbrauch (Std: 10). Max Zeichen/Block: < 300 = schneidet Regeln ab | > 1500 = verstopft Prompt (Std: 800).

98. Automatisches Citation- & Quellen-Pinning (backend/citation_pinner.py)

Problem: Ohne direkte Quellenverweise ist für den Nutzer nicht überprüfbar, aus welchem Dokument eine Faktenaussage des Agenten stammt.

Mechanik: Ordnet Sätze der Antwort per Token-Overlap den abgerufenen Quell-Dokumenten zu, fügt Inline-Belege

([Quelle: ID]) ein und prüft die Mindest-Belegquote.

Nutzen: Macht jede Aussage revisionssicher nachvollziehbar und deckt unbelegte Behauptungen sofort auf.

Preis & Schattenseite (Trade-off): Erhöht den Output-Token-Verbrauch um ~10–15 %; eine zu hohe Mindest-Abdeckung (> 80 %) führt bei kreativen oder zusammenfassenden Fragen zu Fehlwarnungen.

Wann ausschalten? & Schwellenwerte: Ausschalten bei freiem Brainstorming, Code-Generierung oder lockerem Alltags-Chat ohne Dokumentenbezug. | Schwellenwerte: Min. Abdeckung: < 30% = unbelegte Aussagen | > 85% = Fehlwarnungen bei Zusammenfassungen (Std: 60%).

15. Zuverlässige Ausführung & Determinismus (Durable Execution)

99. Structured Outputs & JSON-Schema-Garantie (backend/structured_output_enforcer.py)

Problem: Sprachmodelle liefern manchmal ungültiges JSON (Markdown-Rahmen, fehlende Klammern oder falsche Datentypen), wodurch nachgelagerte Code-Pipelines abstürzen.

Mechanik: Repariert Syntaxfehler im Roh-String automatisch und erzwingt alle Pflichtfelder und Datentypen des übergebenen Ziel-Schemas.

Nutzen: Garantiert 100 % maschinenlesbare, typensichere JSON-Payloads zwischen Agenten und Backend-Schnittstellen.

Preis & Schattenseite (Trade-off): Jede Auto-Reparatur-Schleife kostet +1 bis +3 Sek. Latenz und zusätzliche Tokens. Strikte Schema-Validierung lehnt Antworten auch wegen harmloser Zusatzfelder ab.

Wann ausschalten? & Schwellenwerte: Strikte Validierung ausschalten, wenn API-Endpunkte oder Skripte auch mit flexiblen oder teilweisen JSON-Feldern umgehen können. | Schwellenwerte: Schema-Prüfung: Strikt vs. Tolerant (Std: Strikt). Max Reparaturen: 1–2 = repariert 98 % aller Tippfehler | > 3 = verschwendet Tokens (Std: 2).

100. Durable Execution Checkpointing / Pause & Resume (backend/durable_checkpoint_manager.py)

Problem: Bricht ein 10-schrittiger Recherche- oder Build-Auftrag bei Schritt 8 durch einen Neustart ab, mussten bisher alle 7 Vorarbeiten teuer wiederholt werden.

Mechanik: Sichert nach jedem abgeschlossenen Schritt den kompletten Variablen- und Fortschrittszustand transaktional in SQLite.

Nutzen: Unterbrochene Langläufer setzen exakt am letzten gültigen Checkpoint wieder auf, ohne Arbeit oder Tokens doppelt zu verbrauchen.

Preis & Schattenseite (Trade-off): Checkpointing bei jedem einzelnen Schritt (N = 1) erzeugt spürbare SQLite-Schreiblast. Auto-Resume beim Booten kann veraltete Tasks vom Vortag ungewollt wieder anfeuern.

Wann ausschalten? & Schwellenwerte: Auto-Resume ausschalten, wenn unterbrochene Aufgaben nach einem Neustart erst manuell geprüft und freigegeben werden sollen. | Schwellenwerte: Alle N Schritte: 1 = absolut absturzsicher, mehr Disk-I/O | 3–5 = schont SSD (Std: 1). Aufbewahrung: Std 14 Tage.

101. Idempotency-Keys & Tool-Call-Deduplizierung (backend/idempotency_guard.py)

Problem: Bei Netzwerk-Retries oder Agenten-Schleifen besteht die Gefahr, dass dieselbe E-Mail, Rechnung oder Webhook-Aktion mehrfach ausgelöst wird.

Mechanik: Vergibt für jeden mutierenden Tool-Call einen SHA-256-Idempotency-Key im SQLite-Ledger und blockiert identische Folgeaufrufe innerhalb der TTL.

Nutzen: Verhindert doppelte Außenwirkungen zuverlässig nach dem Vorbild moderner Zahlungs-Gateways (Stripe-Prinzip).

Preis & Schattenseite (Trade-off): Wenn der Nutzer exakt denselben Befehl innerhalb der Sperrfrist (Standard 24 h) absichtlich ein zweites Mal ausführen will, blockiert der Guard.

Wann ausschalten? & Schwellenwerte: TTL-Stunden auf 1 h senken oder kurzzeitig ausschalten, wenn identische Aktionen im Testbetrieb mehrfach hintereinander ausgeführt werden müssen. | Schwellenwerte: TTL (Stunden): < 1h = kurzer Duplikatschutz | > 48h = blockiert gewollte Wiederholungen (Std: 24h).

102. Dry-Run / Shadow-Mode für Agenten-Aufträge (backend/dry_run_simulator.py)

Problem: Neue oder komplexe Agenten-Aufträge möchte man vor der scharfen Ausführung oft erst ohne Risiko an echten Produktionsdaten testen.

Mechanik: Fängt im Shadow-Mode alle schreibenden Datei-, Datenbank- und API-Befehle ab und protokolliert stattdessen einen detaillierten Mutations-Vorschauplan.

Nutzen: Ermöglicht gefahrlose Trockenübungen und Risiko-Audits vor der endgültigen Freigabe.

Preis & Schattenseite (Trade-off): SOLANGE AKTIV, WERDEN KEINE ECHTEN DATEIEN GESCHRIEBEN UND KEINE ECHTEN APIS AUSGEFÜHRT! Vergisst man den Schalter, wirkt das System nach außen hin 'tot'.

Wann ausschalten? & Schwellenwerte: Im normalen Produktivbetrieb zwingend AUSGESCHALTET lassen! Nur gezielt für Trockenübungen (Dry-Runs) und Sicherheits-Audits einschalten. | Schwellenwerte: Hauptschalter standardmäßig AUS (false). Nur für isolierte Testläufe aktivieren.

103. Dead-Letter-Queue (DLQ) mit Auto-Replay-Manager (backend/dead_letter_queue.py)

Problem: Fehlgeschlagene Webhooks oder asynchrone Hintergrund-Jobs gehen bei kurzzeitigen Drittanbieter-Ausfällen ohne Auffangbecken spurlos verloren.

Mechanik: Speichert jeden gescheiterten Job samt Original-Payload und Fehlergrund in der DLQ-Tabelle und wiederholt ihn mit exponentiellem Backoff.

Nutzen: Kein Auftrag geht verloren; fehlgeschlagene Tasks lassen sich jederzeit einzeln oder gesammelt erneut abspielen.

Preis & Schattenseite (Trade-off): Bei dauerhaft fehlerhaften Aufträgen (z. B. ungültiger API-Key) verbrauchen die Wiederholungsversuche unnötig Ressourcen; zeitversetzte Replays können Aktionen verspätet auslösen.

Wann ausschalten? & Schwellenwerte: Auto-Replay ausschalten, wenn fehlgeschlagene Aufträge ausschließlich manuell gesichtet und einzeln freigegeben werden sollen. | Schwellenwerte: Max Retries: 1 = bricht bei kurzem Ausfall ab | > 5 = staut tote Jobs an (Std: 3). Backoff-Basis: Std 10s (10s -> 20s -> 40s).

16. Sicherheit, Datenschutz & Enterprise-Guardrails

104. Bidirektionaler PII-Scrubber & Re-Hydration (backend/pii_scrubber.py)

Problem: Werden E-Mails, Telefonnummern oder IBANs im Klartext an externe Cloud-LLMs übertragen, drohen DSGVO-Verstöße und ungewollter Datenabfluss.

Mechanik: Ersetzt sensible Personendaten vor dem API-Aufruf lokal durch Platzhalter ([EMAIL_1], [IBAN_1]) und tauscht diese in der eintreffenden Antwort wieder gegen die echten Werte aus.

Nutzen: Cloud-Modelle sehen niemals echte Personendaten, während der Nutzer vor Ort dennoch vollständige Dokumente und Antworten erhält.

Preis & Schattenseite (Trade-off): Wenn das LLM den Platzhalter im Antworttext leicht abwandelt, schlägt die Rückübersetzung (Rehydrierung) fehl. Zudem kann Regex-Maskierung manchmal Code-Strings verfälschen.

Wann ausschalten? & Schwellenwerte: Ausschalten, wenn ausschließlich lokale Modelle (Ollama) genutzt werden oder das LLM zwingend die echten E-Mail-Domains/Namen analysieren muss. | Schwellenwerte: Granulare Schalter für E-Mails, Telefonnummern, IBANs und automatische Rehydrierung (alle standardmäßig aktiv).

105. Egress-Firewall & Domain-Allowlisting für Agenten (backend/egress_firewall.py)

Problem: Über versteckte Prompt-Injections in Webseiten könnten Agenten angewiesen werden, interne Daten per HTTP-Request an fremde Angreifer-Server zu senden.

Mechanik: Prüft jede ausgehende Ziel-URL gegen die konfigurierte Domain-Allowlist und blockiert unautorisierte Hosts sowie rohe IPv4-Adressen.

Nutzen: Unterbindet Daten-Exfiltration auf Netzwerkebene selbst bei manipulierten Eingaben.

Preis & Schattenseite (Trade-off): Solange allowlist_only aktiv ist, wird jede neue Webseite, API oder Dokumentation, die nicht explizit in der Domain-Liste steht, bei Web-Recherchen hart blockiert.

Wann ausschalten? & Schwellenwerte: Ausschalten (oder benötigte Domains ergänzen), wenn der Research-Agent frei im offenen Web recherchieren soll. | Schwellenwerte: Erlaubte Domains: Kommagetrennte Whitelist. Roh-IP-Blockade schützt vor direkten IP-Exfiltrationen.

106. Granulare Agenten-RBAC & Datei-Scope-Jails (backend/agent_rbac_jail.py)

Problem: Wenn jeder Sub-Agent vollen Schreibzugriff auf das gesamte Projektverzeichnis besitzt, kann ein Frontend-Agent versehentlich Backend-Kernel-Dateien oder .env-Schlüssel überschreiben.

Mechanik: Bindet jede Agentenrolle an eine feste Werkzeugliste und ein erlaubtes Verzeichnis-Jail (z. B. static/ für Frontend-Agenten).

Nutzen: Setzt das Prinzip minimaler Rechte (Least Privilege) für jeden einzelnen Prozess im Betriebssystem durch.

Preis & Schattenseite (Trade-off): Verhindert, dass ein Administrator-Agent legitim Konfigurationsdateien außerhalb des Projektordners wartet oder .env-Vorlagen direkt überprüft.

Wann ausschalten? & Schwellenwerte: Nur für dedizierte Systemwartungs-Aufgaben kurzzeitig lockern; der .env-Schutz sollte im Normalbetrieb immer aktiv bleiben. | Schwellenwerte: Schalter für Path-Jail, .env-Sperre und Systemordner-Sperre (/etc, /root, /proc).

107. Deterministischer Output-Linter & Policy-Checker (backend/output_policy_linter.py)

Problem: Sprachmodelle verfallen gelegentlich trotz System-Prompt in künstliche KI-Floskeln oder generieren riskante Shell-Snippets.

Mechanik: Scannt den generierten Text in Echtzeit mittels regulärer Ausdrücke, entfernt Sycophancy-Opener/AI-Slop und entschärft destruktive Befehle.

Nutzen: Sichert sprachliche Präzision und technische Sicherheit ohne zusätzlichen LLM-Prüfaufruf.

Preis & Schattenseite (Trade-off): Der Destruktiv-Filter kann auch harmlose Aufräum-Skripte oder gewollte Datenbank-Migrationen blockieren, wenn darin Befehle wie DROP oder rm -rf vorkommen.

Wann ausschalten? & Schwellenwerte: Destruktiv-Blocker kurzzeitig ausschalten, wenn gezielt Server-Aufräumskripte oder SQL-Reset-Migrationen erstellt werden. | Schwellenwerte: Schalter für KI-Floskeln entfernen, Destruktive Befehle blockieren und Auto-Bereinigung.

108. Manipulationssichere Execution Receipts / SHA-256-Quittungen (backend/execution_receipt_ledger.py)

Problem: Für Compliance-Audits (EU AI Act / ISO 42001) reichen einfache Text-Logs nicht aus, da sie nachträglich verändert werden können.

Mechanik: Verkettet jede Task-Abrechnung (Modell, Tokens, USD-Kosten, ausgeführte Tools) mit dem SHA-256-Hash des Vorgängereintrags in einer Blockchain-ähnlichen Kette.

Nutzen: Jede nachträgliche Manipulation an Kosten- oder Entscheidungsprotokollen fällt bei der Integritätsprüfung sofort auf.

Preis & Schattenseite (Trade-off): Verursacht minimalen Rechen- und Speicher-Overhead (~2 ms und ~2 KB SQLite-Speicher pro Task); die Hash-Kette wächst über Monate kontinuierlich an.

Wann ausschalten? & Schwellenwerte: Kann im Produktivbetrieb dauerhaft aktiviert bleiben; nur auf extrem speicherlimitierten Kleinstsystemen ausschalten. | Schwellenwerte: Hash-Verkettung (SHA-256) + detaillierte Kosten-Aufschlüsselung pro Lauf.

17. Selbst-Optimierung, Evaluation & Schwarm-Intelligenz

109. Automatischer Prompt-Optimizer nach DSPy/TextGrad-Prinzip (backend/prompt_optimizer.py)

Problem: Statische System-Prompts wiederholen dieselben Fehler immer wieder, solange kein Mensch manuell nachbessert.

Mechanik: Wertet reale Fehler-Logs einer Rolle aus, erkennt systematische Schwächen und generiert gezielte Guardrail-Zusatzregeln für den System-Prompt.

Nutzen: Agenten lernen kontinuierlich aus gescheiterten Läufen und härten ihre Instruktionen selbstständig.

Preis & Schattenseite (Trade-off): Ist Auto-Apply aktiv, kann eine fehlerhaft abgeleitete Regel das Verhalten des Agenten dauerhaft verzerren (Prompt-Drift) und den System-Prompt aufblähen.

Wann ausschalten? & Schwellenwerte: Auto-Apply im Produktivbetrieb ausgeschaltet lassen, damit vorgeschlagene Prompt-Patches erst nach menschlicher Prüfung übernommen werden. | Schwellenwerte: Ab Fehlern: 1 =

überreagiert auf Einzelfehler | > 8 = lernt sehr träge (Std: 3). Max Regeln: > 10 = bläht System-Prompt auf (Std: 5).

110. Gold-Standard Regression-Test-Suite / Eval-Runner (backend/eval_regression_runner.py)

Problem: Eine vermeintlich kleine Änderung am System-Prompt oder Modell kann bestehende Sicherheits- oder Formatregeln unbemerkt verschlechtern.

Mechanik: Führt vor der Übernahme neuer Einstellungen automatisierte Gold-Standard-Testfälle aus und prüft Bestehensquote sowie verbotene Muster.

Nutzen: Verhindert Qualitäts-Regressionen im Produktivbetrieb durch ein objektives Quality-Gate.

Preis & Schattenseite (Trade-off): Jeder komplette Eval-Durchlauf feuert alle hinterlegten Test-Prompts gegen die LLM-API (kostet je nach Testumfang ~5.000–25.000 Tokens und 15–60 Sek. Laufzeit).

Wann ausschalten? & Schwellenwerte: Deploy-Blockade ausschalten, wenn schnelle experimentelle Prompt-Tests ohne Wartezeit durchgeführt werden. | Schwellenwerte: Min. Pass-Rate: < 75% = lässt Verschlechterungen durch | 100% = blockiert schon bei kleinsten Abweichungen (Std: 90%).

111. Multi-Armed Bandit Modell-Router (backend/bandit_model_router.py)

Problem: Feste Modell-Zuweisungen reagieren nicht darauf, wenn ein Cloud-Provider plötzlich langsamer wird oder erhöhte Fehlerraten zeigt.

Mechanik: Bepunktet jedes Modell laufend nach realer Erfolgsquote (60 %) und Geschwindigkeit (40 %) und wählt dynamisch den besten Kandidaten.

Nutzen: Maximiert Antwortqualität und Tempo automatisch anhand echter Laufzeit-Telemetrie.

Preis & Schattenseite (Trade-off): Durch die Explorationsrate (Epsilon, z. B. 0.15 = 15 %) testet der Router absichtlich immer wieder alternative Modelle aus, die im Einzelfall langsamer sein können.

Wann ausschalten? & Schwellenwerte: Ausschalten (oder Epsilon auf 0.00 setzen), wenn für jeden Lauf ein 100 % festes, vorhersagbares Modell erzwungen werden soll. | Schwellenwerte: Epsilon: 0.00 = nutzt stur das beste Modell | > 0.30 = probiert zu oft schwächere Modelle (Std: 0.15). Erfolgs-/Latenz-Gewicht: Std 0.60 / 0.40.

112. Dynamisches Ephemeral-Team-Spawning (backend/ephemeral_team_spawner.py)

Problem: Ein einzelner Generalisten-Agent ist bei großen Querschnittsaufgaben (Architektur + UI + Security + QA) schnell überfordert.

Mechanik: Rekrutiert passend zum Missions-Briefing eine temporäre Spezialisten-Task-Force und löst sie nach erledigter Mission wieder auf.

Nutzen: Bündelt Spezialwissen bedarfsgerecht ohne dauerhaften Ressourcen-Overhead.

Preis & Schattenseite (Trade-off): Ein 4-köpfiges Ad-hoc-Team vervielfacht den Token-Verbrauch (3x bis 5x) und benötigt zusätzlichen Koordinations-Overhead zwischen den Teammitgliedern.

Wann ausschalten? & Schwellenwerte: Ausschalten bei einfachen Einzelaufgaben oder knappem Token-Budget; Max. Teamgröße im Alltag auf 3–4 begrenzen. | Schwellenwerte: Max Teamgröße: 2–4 = ideal | > 5 = hoher Kommunikations-Overhead & 5x Token-Verbrauch (Std: 4).

113. Live-Ressourcen-Protector & Host-Backpressure (backend/resource_backpressure.py)

Problem: Parallele Container, Browser-Audits und lokale Prozesse können bei Lastspitzen den Host-RAM oder die CPU überlasten (OOM-Kill).

Mechanik: Überwacht CPU-, RAM- und Disk-Metriken in Echtzeit und drosselt niedrig priorisierte Hintergrund-Tasks ab 85 % Auslastung automatisch.

Nutzen: Schützt den Server zuverlässig vor Abstürzen durch Überlastung und hält kritische Kernprozesse jederzeit reaktionsfähig.

Preis & Schattenseite (Trade-off): Sind die Schwellenwerte zu niedrig eingestellt (CPU < 70 % oder RAM < 70 %), werden Hintergrund-Aufgaben schon bei normalen kurzen Lastspitzen ständig ausgebremst.

Wann ausschalten? & Schwellenwerte: Nicht komplett ausschalten — stattdessen bei starker Hardware die CPU-/RAM-Schwellen auf 90 % anheben. | Schwellenwerte: Max CPU/RAM: < 70% = drosselt bei normaler Arbeit | > 92% = OOM-Gefahr (Std: 85%). Min. freie Disk: Std 2.0 GB.

18. Säule 3: Wirtschaft & ERP (alpha) & Mandanten-Suite

Modul / View	Funktionsumfang & Technische Merkmale (v0.5.1)
Adressen & CRM + PocketBase-Sync	Zentraler Kontakt-, Debitoren- (10000+) und Kreditorenstamm (70000+), 360°-Kontakt-Drawer, Multi-Channel-Timeline, Kalender/Wiedervorlagen sowie LinkedIn- & Medien-Sync über PocketBase (Ports 3005/8090/8092).
Artikel & Leistungen (ART)	Stammdatenkatalog ab ART-1001 für Dienstleistungen, Stundensätze, Pauschalen, SaaS-Abonnements sowie physische Produkte mit Lagerbestandsführung.
Vorgänge & Verträge	Durchgängige Belegkette Angebot (ANG-YYYY-NNNN) → Auftrag (AB-YYYY-NNNN) inkl. gewichteter Deal-Pipeline, Umsatz-Forecast und Vertrags-Fristenradar (30/14/7-Tage-Alarme).
Rechnungswesen & DATEV	Ausgangs- und Eingangsbelege nach § 14 UStG, GiroCode-Zahlungs-QR, Finanz-Analytics, automatisches Mahnwesen (Dunning Engine) und EXTF/CSV-Export für DATEV.
Corporate & Mandanten-Isolation	Mehrere eigene Unternehmungen (company_id) mit physisch getrennten SQLite-Datenbanken im WAL-Modus, eigenen Nummernkreisen, Briefbögen und Vorlagen-Editor.

19. Architektur-, Security- & Deployment-Features (v0.5.0 – v0.5.1)

Feature / Subsystem	Implementierung & Betriebsnutzen (v0.5.1)
Modulare Router-Architektur	Auslagerung in dedizierte FastAPI-Router unter archon/routers/ (main.py von 16.254 auf 1.875 Zeilen reduziert, -88 %) und zentrale Konfigurationsschicht archon/core/config.py.
Security Hardening (Archon Hardened)	Schließung von 25 kritischen und 68 hohen Audit-Befunden; Starlette Threadpool-Entlastung, gehärtete UI-DOM-Pfade und Klasse-A Docker-Admin-Proxy für hermes_updater.
RBAC-Benutzerverwaltung (17.10)	Granulare Rollen (admin, operator, viewer) über backend/auth_service.py mit 100.000 Runden PBKDF2-HMAC-SHA256 Passwort-Hashing und Single-User/Multi-User-Modus.
2-Phasen Systemupdate (17.11)	Sofortige Anzeige lokaler Version/HEAD via ?local_only=true (~0,25 s) plus asynchroner GitHub-Abgleich, Pre-Update SQLite Online-Snapshots und 1-Klick-Rollback.
AnythingLLM Proxy & Chroma Stack	Nahtlose Einbettung von AnythingLLM in die Zentralbibliothek über /anythingllm-proxy inkl. /assets/* Fast-Path und lokalem ChromaDB Vektor-Index.
Portabler 1-Click-Installer	Plattformübergreifende Erstinstallation über setup.bat, setup.ps1 und install.sh mit docker-compose.yml, .env.example, INSTALL_AGENT.md und 9-sprachiger GitHub-Doku.
4-Augen Qualitäts-Gate & Code-Sandbox	Automatische Zweitprüfung im Kanban vor Ticket-Abschluss sowie interaktive Code-Sandbox im Showroom (4.4.1) zur Live-Vorschau erzeugter Artefakte.

20. Alphabetisches Glossar & Stichwortverzeichnis

Alphabetisches Begriffsverzeichnis aller Kernmodule, Architekturkomponenten und Features (v0.5.1) mit exakten Seitenzahlen.

Feature / Begriff	Kurzbeschreibung	Seite(n)
3-Tier Kosten-Routing	Automatische Modellauswahl in LOW / MEDIUM / HIGH nach Aufgabenkomplexität und Budget.	S. 3, 8
4-Augen Qualitäts-Gate	Unabhängige Prüfung abgeschlossener Kanban-Aufgaben durch einen Revisor-Agenten.	S. 3, 14, 31
Anti-AI-Slop & STE100	Stil- und Wortfilter zur Vermeidung unnötiger Floskeln und Einsparung von Output-Tokens.	S. 8, 10
AnythingLLM Proxy	Lokale Einbettung von AnythingLLM + ChromaDB über /anythingllm-proxy in der Zentralbibliothek.	S. 3, 31
Approval Gate (HITL)	Menschlicher Prüfschritt vor externen E-Mails, Buchungen oder kritischen Dateizugriffen.	S. 4, 17, 22, 28
Archon Hardened (v0.5.1)	Sicherheits- und Stabilitäts-Release mit modularer Router-Struktur und geschlossenen Audit-Befunden.	S. 3, 31
Audit-Log (SHA-256 Hashkette)	Revisionssichere Protokollkette aller System- und Agentenaktionen.	S. 13, 17, 28, 29
Bandit Modell-Router	Adaptiver Router (Multi-Armed Bandit) zur dynamischen Modellwahl nach Trefferquote und Kosten.	S. 29, 30
Bot Screen (Web-Navigator)	Visueller Ausführungsmonitor für Browser-Interaktionen via Playwright.	S. 14, 23
ChromaDB	Lokale Vektordatenbank für semantisches Dokumenten-Retrieval in Säule 2.	S. 25, 31
Context-Compaction & LRU-K	Speicherauslagerung und Komprimierung langer Chat-Kontexte bei Erreichen des RAM-Limits.	S. 8, 9, 24, 25
Corporate & Mandantenfähigkeit	Physische Datenbank-Trennung mehrerer Gesellschaften per company_id.	S. 3, 31
Corrective RAG (CRAG)	Selbstprüfende Wissensabfrage mit Relevanzbewertung vor Antwortgenerierung.	S. 25, 26
DAG-Scheduler	Zweistufige Ausführungsplanung abhängiger Teilaufgaben mit parallelem Fan-Out / Fan-In.	S. 12, 13, 24
DATEV-Export & GiroCode	EXTF/CSV-Schnittstelle für Buchhaltungsdaten und EPC-Zahlungs-QR auf Rechnungen.	S. 3, 31
Dead-Letter-Queue (DLQ)	Fehlerwarteschlange mit automatischer Wiederholungssteuerung für gescheiterte Jobs.	S. 27, 28
Decision Engine (55 Frameworks)	Strategiemodul für strukturierte Boardroom-Debatten über 9 Methodik-Kategorien.	S. 3, 5, 20
DSPy / TextGrad Optimizer	Automatische Verbesserung von System-Prompts anhand realer Evaluations-Metriken.	S. 29
Durable Execution & Time-Travel	Unterbrechbare Langläufe mit SQLite-Checkpoints und schrittweisem Execution-Replay.	S. 3, 21, 27, 28
Egress-Firewall	Ausgehende Netzwerk- und Domain-Kontrolle für Agenten-Toolaufrufe.	S. 28

Feature / Begriff	Kurzbeschreibung	Seite(n)
Ephemeral Code-Sandbox	Flüchtige, isolierte Container-Laufzeit für sicheren Python- und Skriptcode.	S. 3, 23, 31
ERP-Belegkette (ANG -> AB -> RE)	Verknüpfte Vertriebs- und Buchhaltungsdokumente inkl. Pipeline-Forecast und Mahnwesen.	S. 22, 24, 25, 31
Event-Mesh & Webhook-Trigger	Ereignisgesteuerte Verarbeitung eingehender Webhooks und Ordneränderungen.	S. 14, 21, 22
Evey / Hermes Plugins	Modulare System-Erweiterungen für Speicher, Delegation, Verifikation und Schnittstellen.	S. 3, 19, 20, 21
Hermes Master Orchestrator	Zentrale Kern-Engine für Zielzerlegung, Delegation und Synthese im Daemon auf Port 8420.	S. 9, 11, 12, 19
Hybrid-Search (FTS5 + RRF)	Kombination aus BM25-Volltextsuche und Vektorsuche per Reciprocal Rank Fusion.	S. 25
Idempotency-Keys & Dry-Run	Schutz vor doppelter Tool-Ausführung sowie Simulationsmodus ohne Schreibzugriff.	S. 27, 28
JEV Router (0-Token Intent)	Deterministischer Vorfilter ohne KI-Kosten für eindeutige Kurzbefehle.	S. 8, 10
Knowledge-Graph	Lokaler Tripel-Speicher für strukturierte Entitätsbeziehungen neben Vektordaten.	S. 25, 26
MCP (Model Context Protocol)	Standardisierte Treiberarchitektur zur sicheren Werkzeuganbindung.	S. 6, 14
OmniRoute AI Gateway	Multi-Provider-Gateway inkl. Free-Tier-Routing, Latency-Hedging und Eval-Arena.	S. 3, 6, 21
PBKDF2 / Argon2id RBAC	Rollen- und Passwort-Absicherung im System Control Center (Tab 17.10).	S. 3, 28, 29, 31
PII-Scrubber & Re-Hydration	Bidirektionale Anonymisierung sensibler Personendaten vor externen LLM-Calls.	S. 3, 28
Pre-Flight Kostenschätzer	Vorabberechnung erwarteter Token-Kosten inkl. Task-Budget-Stopp bei Überschreitung.	S. 8, 10, 11
Pre-Mortem & Konsens-Gate	Simulierter Fehler-Stresstest und Mehrfach-Agenten-Abstimmung vor kritischen Freigaben.	S. 5, 21, 23, 24
Research Lab	Mehrstufige Web- und Quellenrecherche mit Evidenz-Triangulation.	S. 3, 7, 21
Router-Architektur (archon/routers)	Modulare Backend-Aufteilung in schlanke FastAPI-Router (-88 % Dateigröße in main.py).	S. 3, 31
Semantic Caching	24h-Zwischenspeicherung inhaltlich ähnlicher Prompts zur Token-Einsparung.	S. 8, 9, 24, 26
SLA-Verträge (Contract Enforcement)	Formale Übergabe- und Qualitätszusicherungen zwischen spezialisierten Agenten.	S. 13, 23, 24
Sleep-Cycle Konsolidierung	Nächtliche Deduplizierung und Langzeit-Verdichtung des Agenten-Gedächtnisses.	S. 15, 23
SQLite WAL Connection-Pool	Schreibsicherer Multi-Thread-Datenbankzugriff über Write-Ahead Logging.	S. 15, 20, 21, 24
System Control Center (11 Tabs)	Zentrale Administrationskonsole inkl. Benutzer/RBAC und 2-Phasen Systemupdate.	S. 3, 8, 11, 31

Feature / Begriff	Kurzbeschreibung	Seite(n)
Token Guard & Loop-Killer	Echtzeit-Schutz gegen Endlosschleifen (Jaccard-Prüfung) und Token-Überschreitungen.	S. 3, 6, 8, 11
Watchdog & Deadlock-Supervisor	30-Sekunden-Überwachung hängender Tasks, Container und automatischer Selbstheilung.	S. 6, 11, 12, 21